

UNIVERSIDAD AUTONOMA DE CHIHUAHUA

Bases de Datos Avanzadas

Proyecto 2a: BD Autos en MongoDB

Profesor: JOSE SAUL DE LIRA MIRAMONTES

Alumnos: LEONARDO FRANCO BULKLEY
IVAN EDUARDO URRUTIA RODRIGUEZ
JONATHAN GANDARA SALAZAR

04/11/2026

I. Modelado de la base de datos en MongoDB

Las 6 tablas de BDAutosU2 quedaron en 3 colecciones. Embebí lo que siempre se lee junto y dejé enlazado (linking) lo que se consulta por separado.

Colección	Tablas	Diseño
continents	Continents, Countries	Los países van embebidos en el continente (1:N).
makers	Car_Makers, Model_Details	El país con su continente va embebido (referencia extendida) y las marcas van embebidas en el arreglo models (1:N).
cars	Car_Names, Car_Details	Las especificaciones van embebidas en details (1:1). La marca se copia en model y el fabricante se enlaza con makerId.

Ejemplo de cada colección:

```
continents { _id: 2, name: "europe", countries: [ { _id: 2, name: "germany" }, ... ] }

makers { _id: 3, name: "stellantis", fullName: "Stellantis N.V.",
         country: { _id: 17, name: "netherlands", continent: { _id: 2, name: "europe" } },
         models: [ { _id: 7, name: "jeep" }, { _id: 8, name: "dodge" }, ... ] }

cars { _id: 1, descr: "chevrolet silverado 1500 lt 5.3",
       model: { _id: 1, name: "chevrolet" }, makerId: 1,
       details: { mpg: 18, cylinders: 8, edispl: 325, horsepower: 355,
                  weight: 5100, accel: 6.7, year: 2024 } }
```

Schemas

Cada colección tiene su schema como clase de Python en app/modules/<colección>/schemas/, igual que el @Schema() de NestJS: una clase para el documento y una por cada subdocumento embebido. Así quedó el de cars:

```
@dataclass
class CarModel:
    _id: int
    name: str


@dataclass
class Details:
    mpg: float
    cylinders: int
    edispl: int
    horsepower: int
    weight: int
    accel: float
    year: int


@dataclass
class Car:
    _id: int
```

```
    descr: str
    model: CarModel
    makerId: int
    details: Details


CAR_INDEXES = [({"makerId": 1}, {}), ({"model._id": 1}, {})]
```

Hice índices en cars.makerId y cars.model._id porque son los campos de los $lookup, y un índice único en makers.models.name que equivale al model_unique de Oracle. Los índices se declaran en el mismo archivo del schema.

II. Script de la base de datos

El script scripts/migrate.py lee los INSERT de BDAutosU2.sql, arma los documentos con las clases de los schemas, carga las 3 colecciones y genera database/autos_mongodb.js. Ese archivo crea la base completa sin Python: borra cada colección, inserta sus documentos y crea sus índices.

```
db.continents.insertMany([

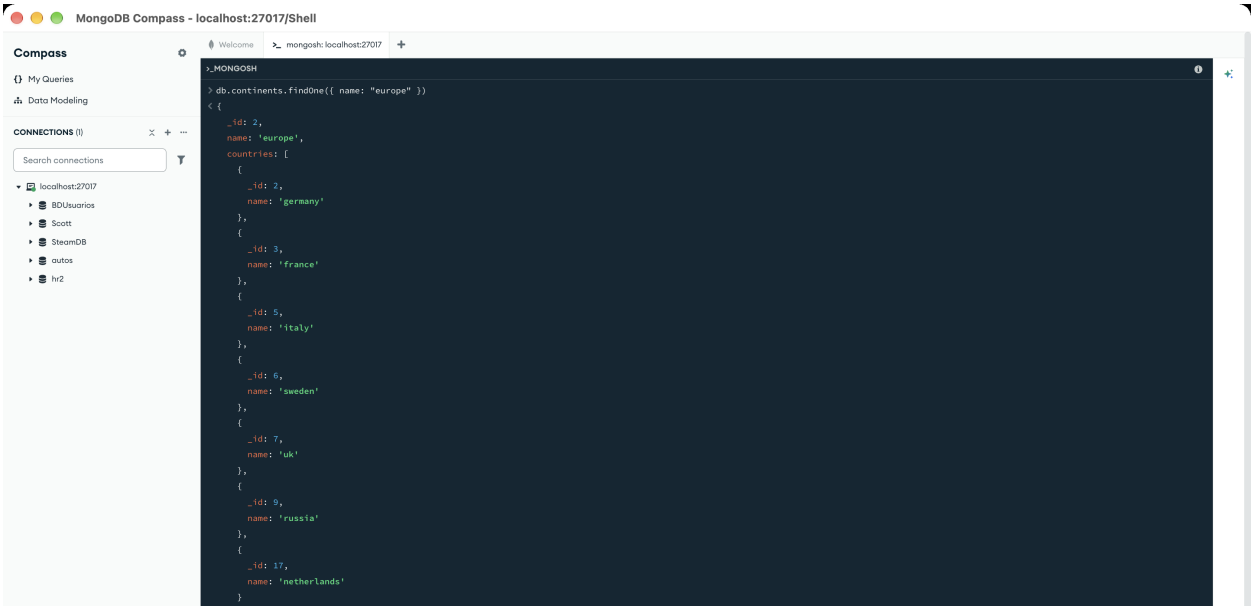
db.continents.drop();
db.continents.insertMany(...);
db.makers.drop();
db.makers.insertMany(...);
db.makers.createIndex(...);
db.cars.drop();
db.cars.insertMany(...);
db.cars.createIndex(...);
db.cars.createIndex(...);
```

Lo corrí con mongosh database/autos_mongodb.js y luego revisé las colecciones, el conteo de documentos y los índices:

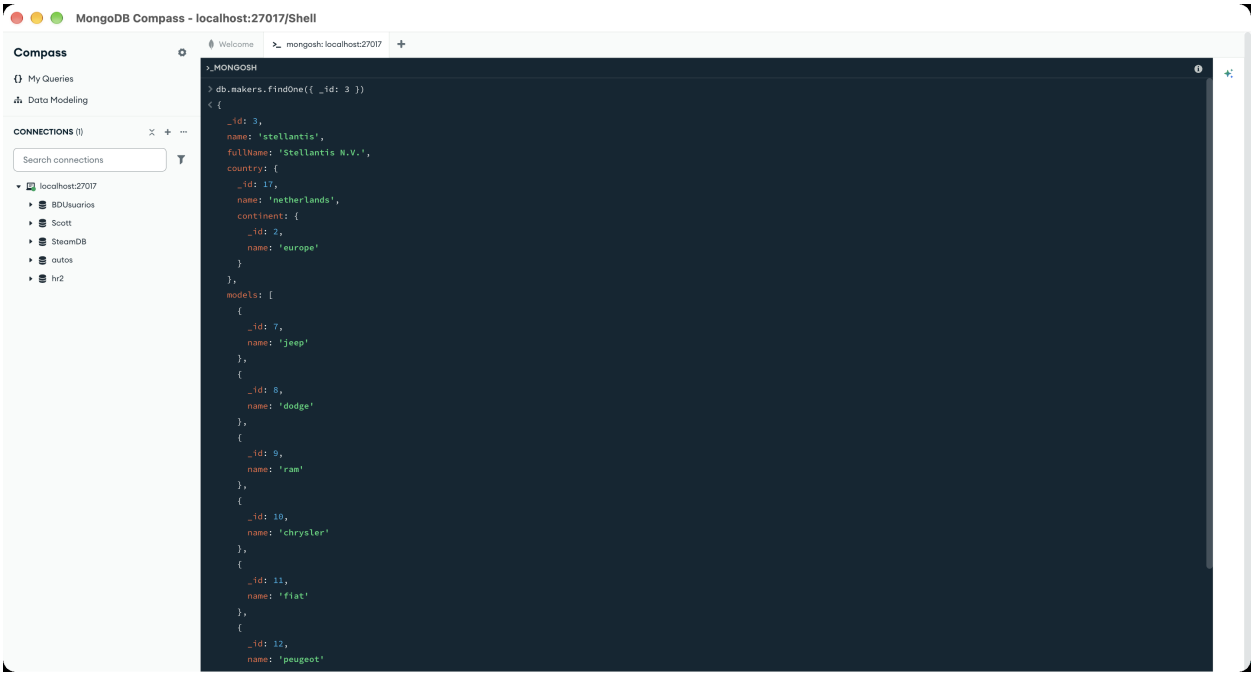
```
> show collections
cars
continents
makers
> db.continents.countDocuments()
5
> db.makers.countDocuments()
20
> db.cars.countDocuments()
138
> db.makers.getIndexes().map(i => i.key)
[
  {
    _id: 1
  },
  {
    'models.name': 1
  }
]
> db.cars.getIndexes().map(i => i.key)
[
  {
    _id: 1
  },
  {
    makerId: 1
  },
  {
    'model._id': 1
  }
]
```

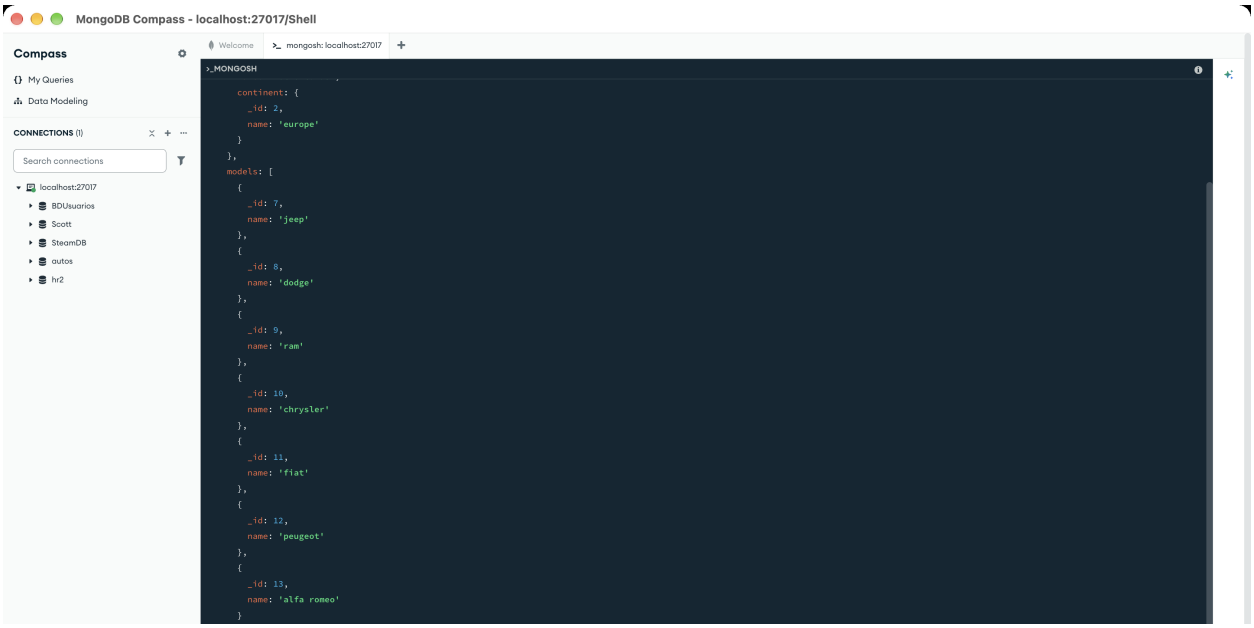
Quedan 5 continentes, 20 fabricantes y 138 autos, igual que en Oracle.

Documento de continents

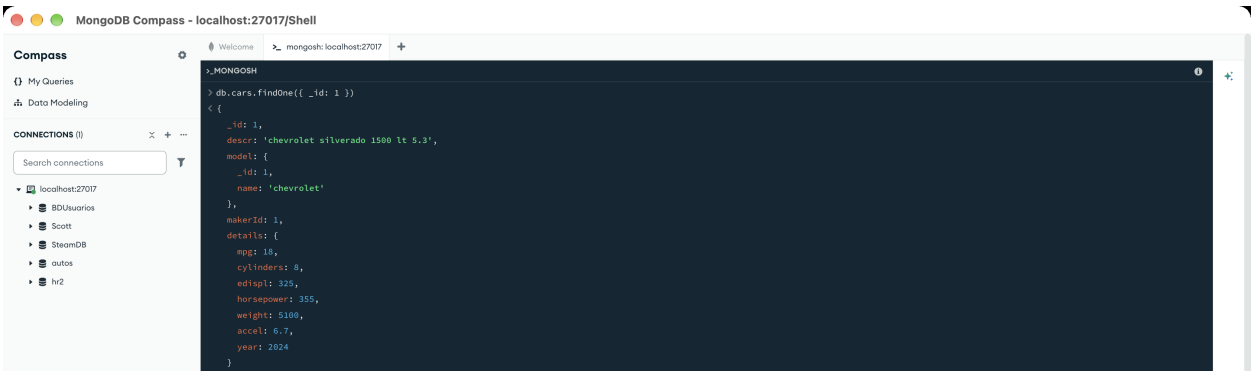


Documento de makers





Documento de cars

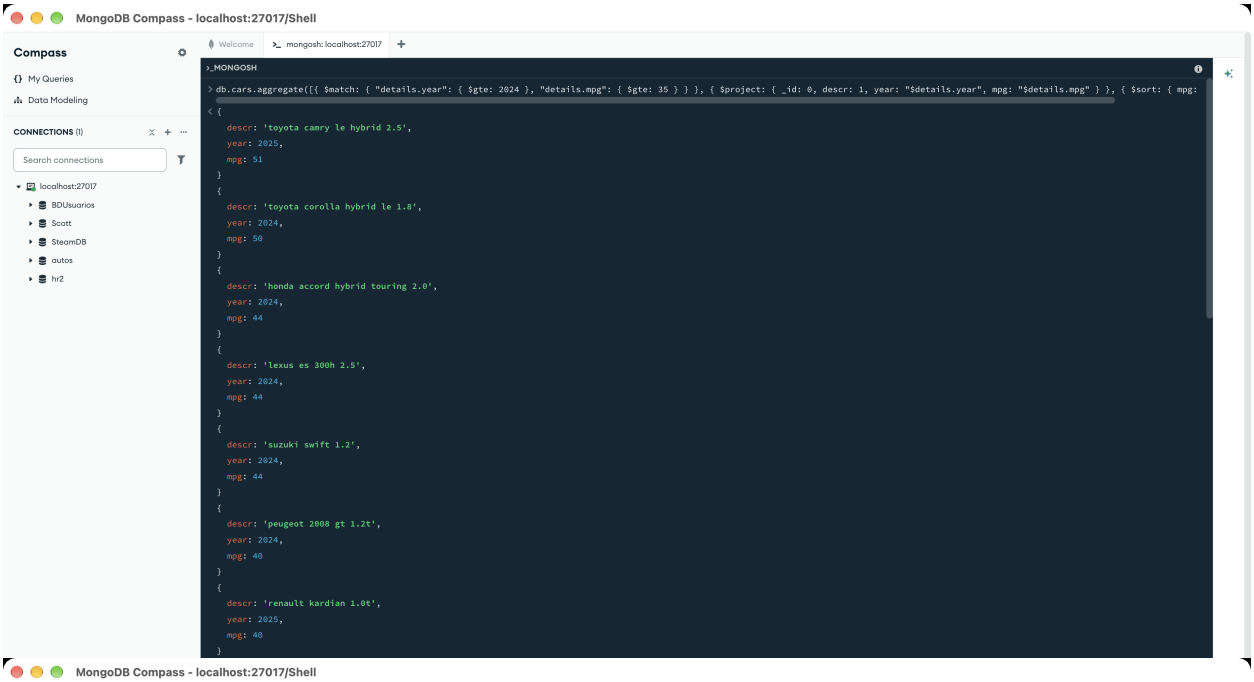


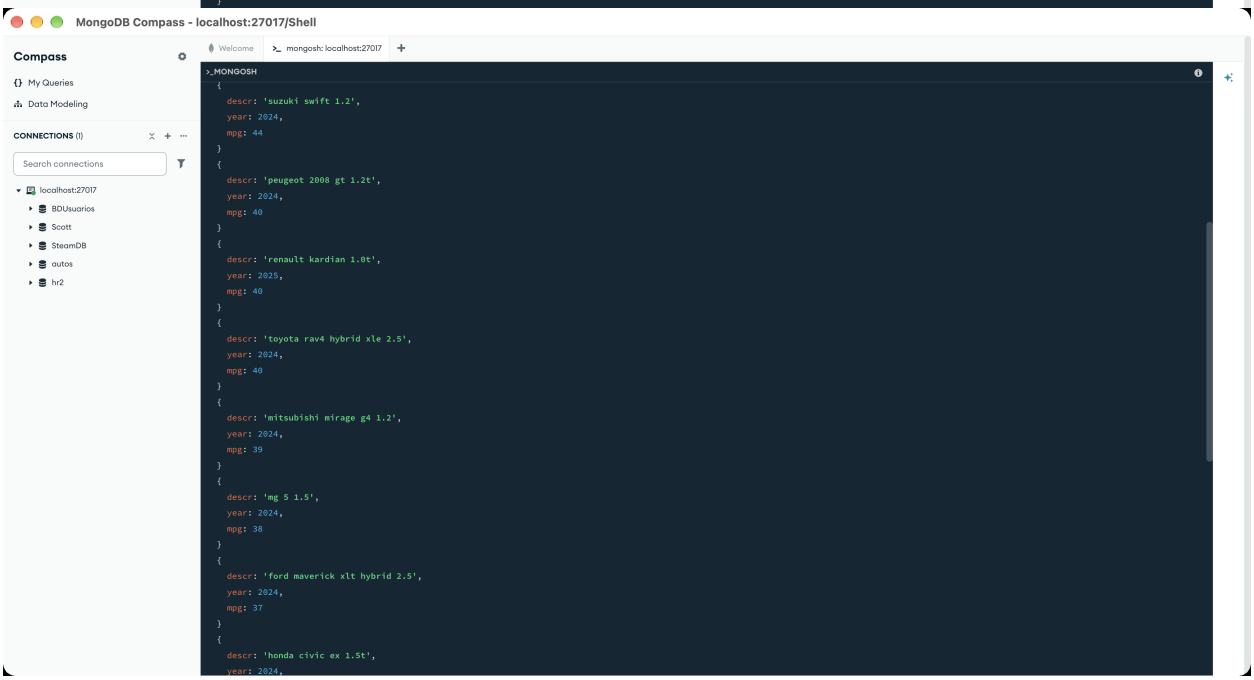
III. Consultas

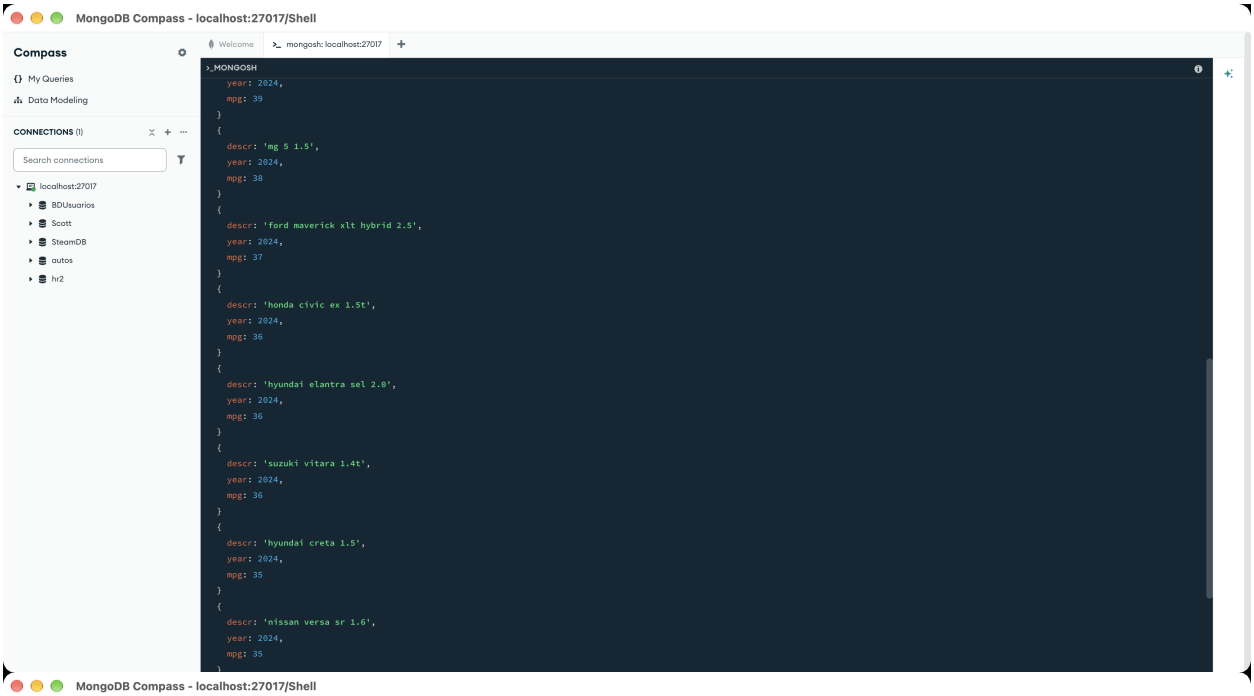
Cada consulta es un método queryN() del service de la colección donde empieza. Las capturas son del shell de Compass; el código de las 25 también está en database/queries.js.

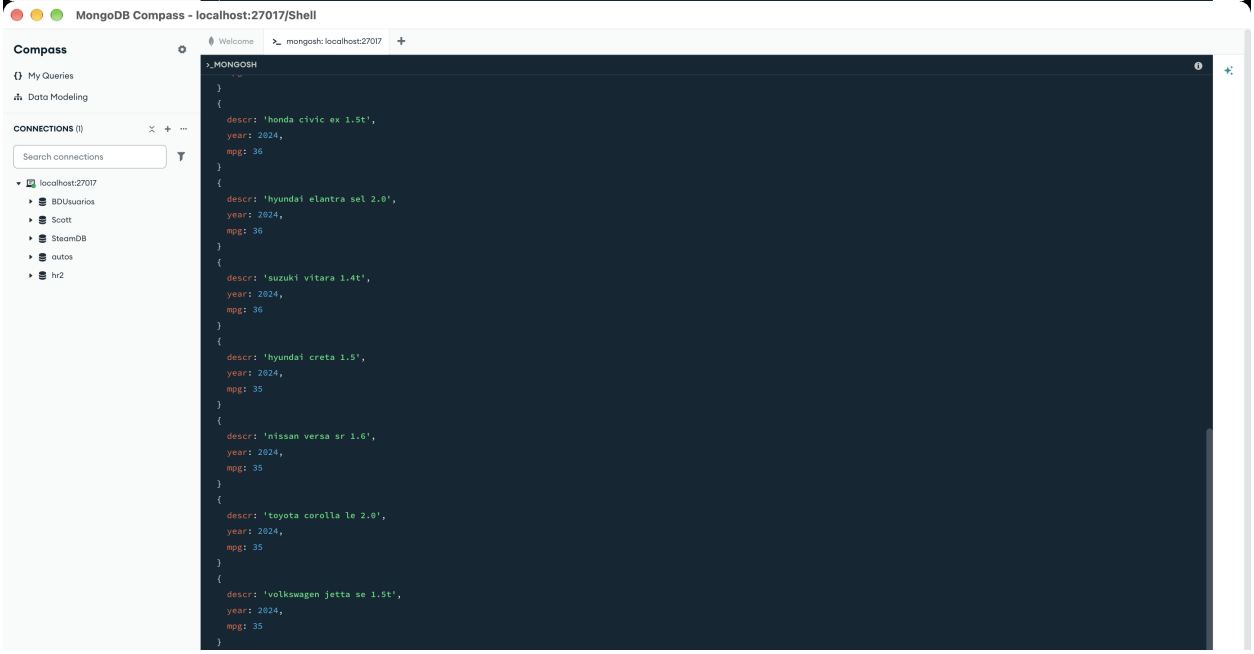
1. Obtener los autos de modelo 2024 en adelante con rendimiento de al menos 35 mpg. Mostrar solo la descripción, el año y el rendimiento, ordenados de mayor a menor rendimiento.

Método: CarsService.query1(), colección cars



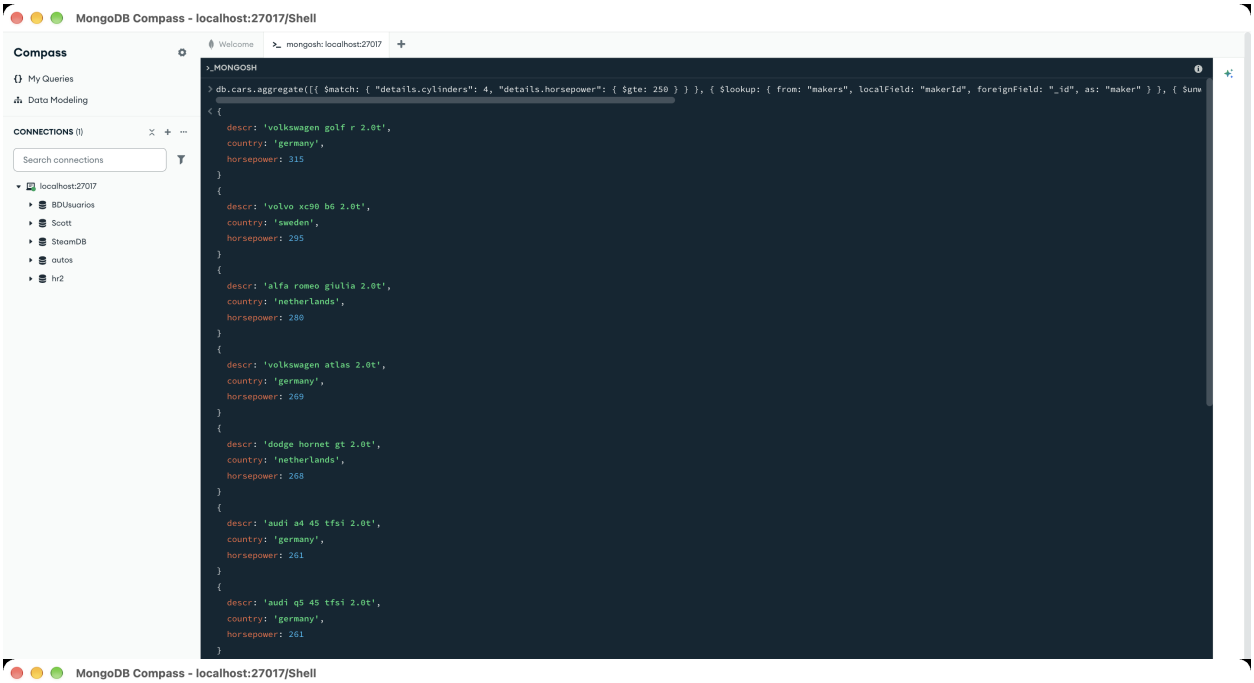


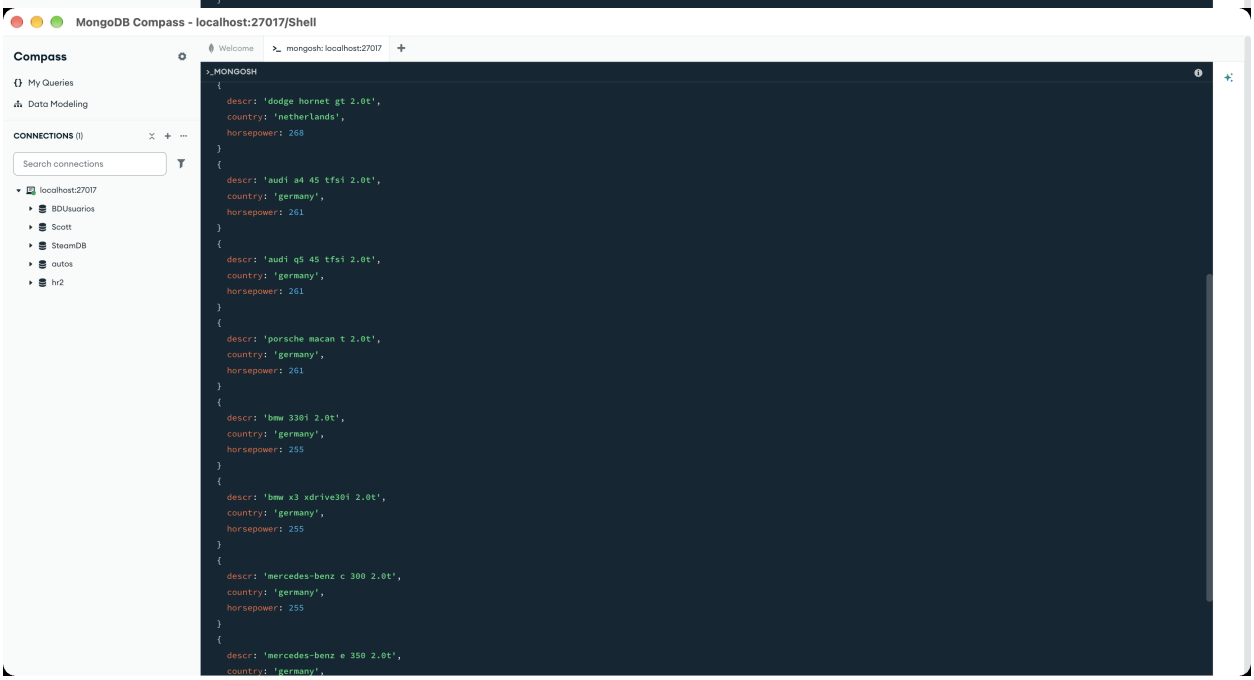

MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
year: 2024,
mpg: 39
descr: 'mg 5 1.5',
year: 2024,
mpg: 38
descr: 'ford maverick xlt hybrid 2.5',
year: 2024,
mpg: 37
descr: 'honda civic ex 1.5t',
year: 2024,
mpg: 36
descr: 'hyundai elantra sel 2.0',
year: 2024,
mpg: 36
descr: 'suzuki vitara 1.4t',
year: 2024,
mpg: 36
descr: 'hyundai creta 1.5',
year: 2024,
mpg: 35
descr: 'nissan versa sr 1.6',
year: 2024,
mpg: 35

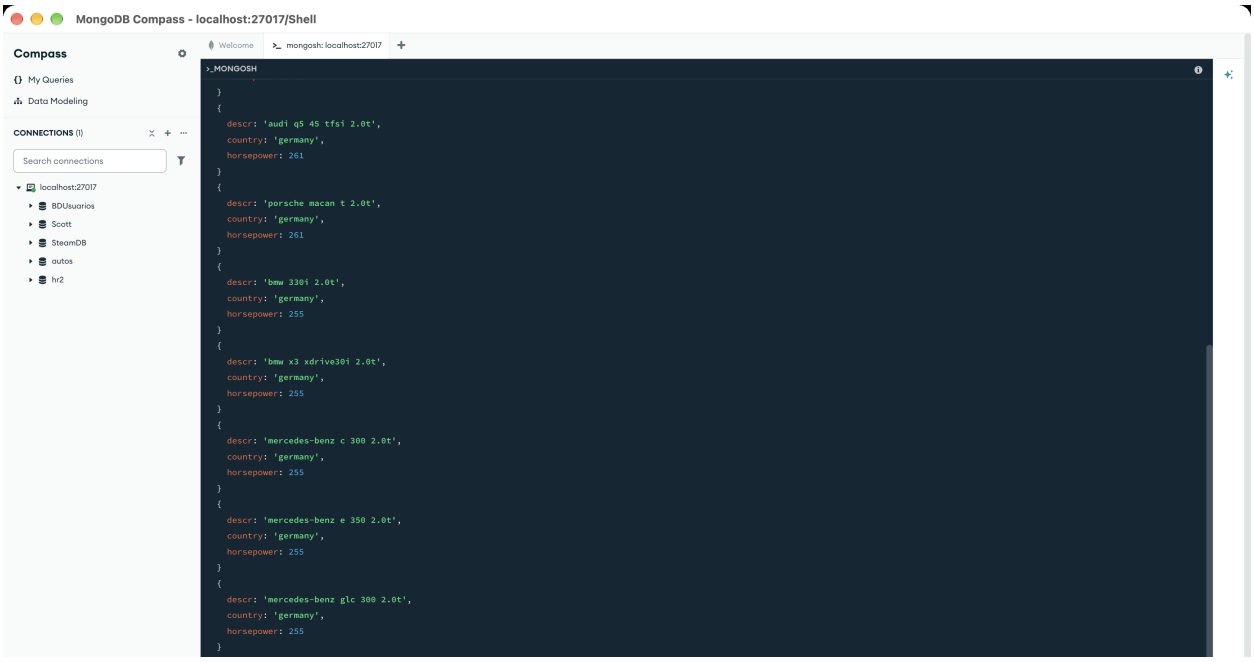

MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
descr: 'honda civic ex 1.5t',
year: 2024,
mpg: 36
descr: 'hyundai elantra sel 2.0',
year: 2024,
mpg: 36
descr: 'suzuki vitara 1.4t',
year: 2024,
mpg: 36
descr: 'hyundai creta 1.5',
year: 2024,
mpg: 35
descr: 'nissan versa sr 1.6',
year: 2024,
mpg: 35
descr: 'toyota corolla le 2.0',
year: 2024,
mpg: 35
descr: 'volkswagen jetta se 1.5t',
year: 2024,
mpg: 35

2. Listar los autos de fabricantes europeos con motor de 4 cilindros y al menos 250 hp, mostrando descripción, país y potencia, ordenados por potencia descendente.

Método: CarsService.query2(), colección cars

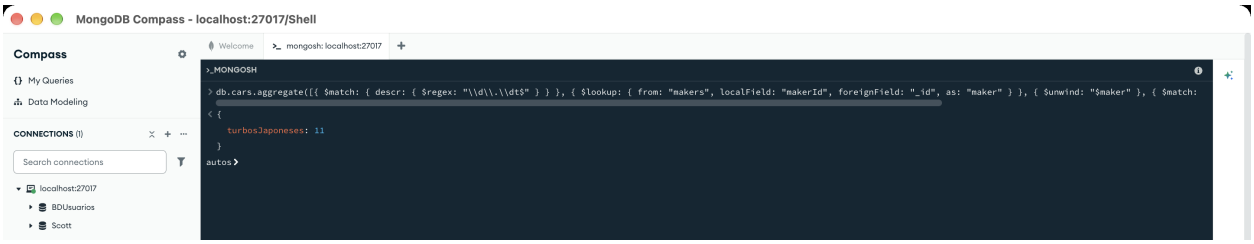






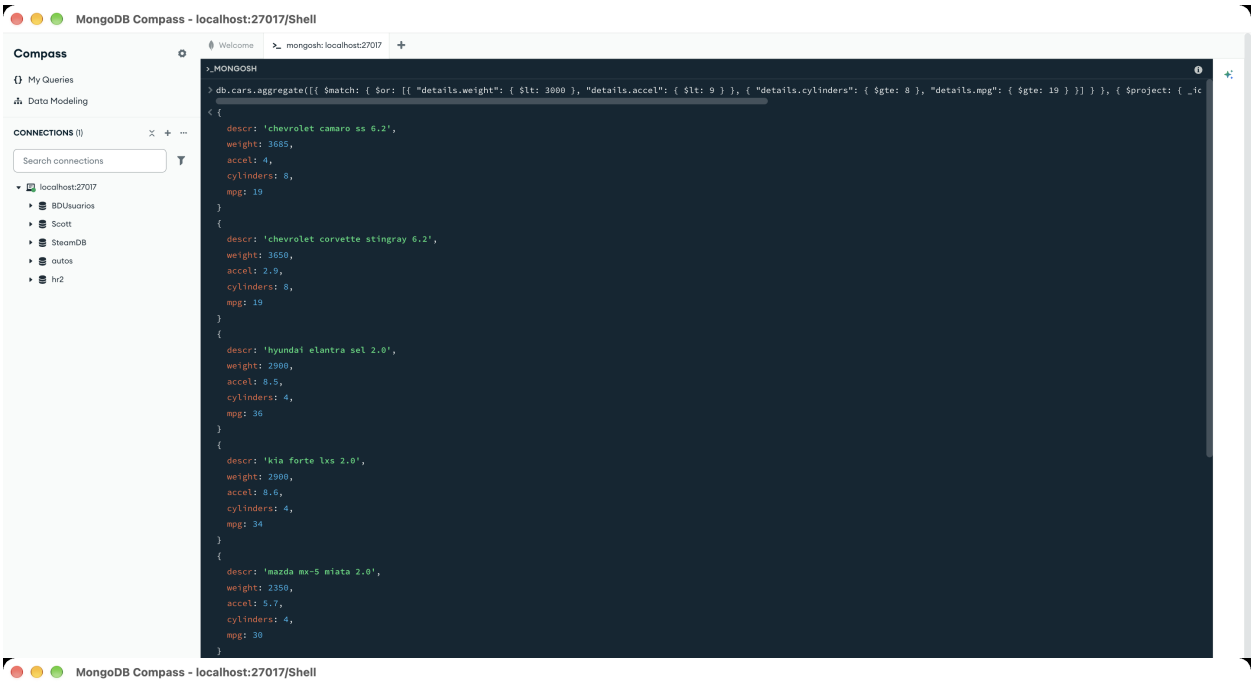
3. Contar cuántos autos de fabricantes japoneses tienen motor turbo, es decir, cuya descripción termina con la cilindrada seguida de "t" (por ejemplo "1.5t").

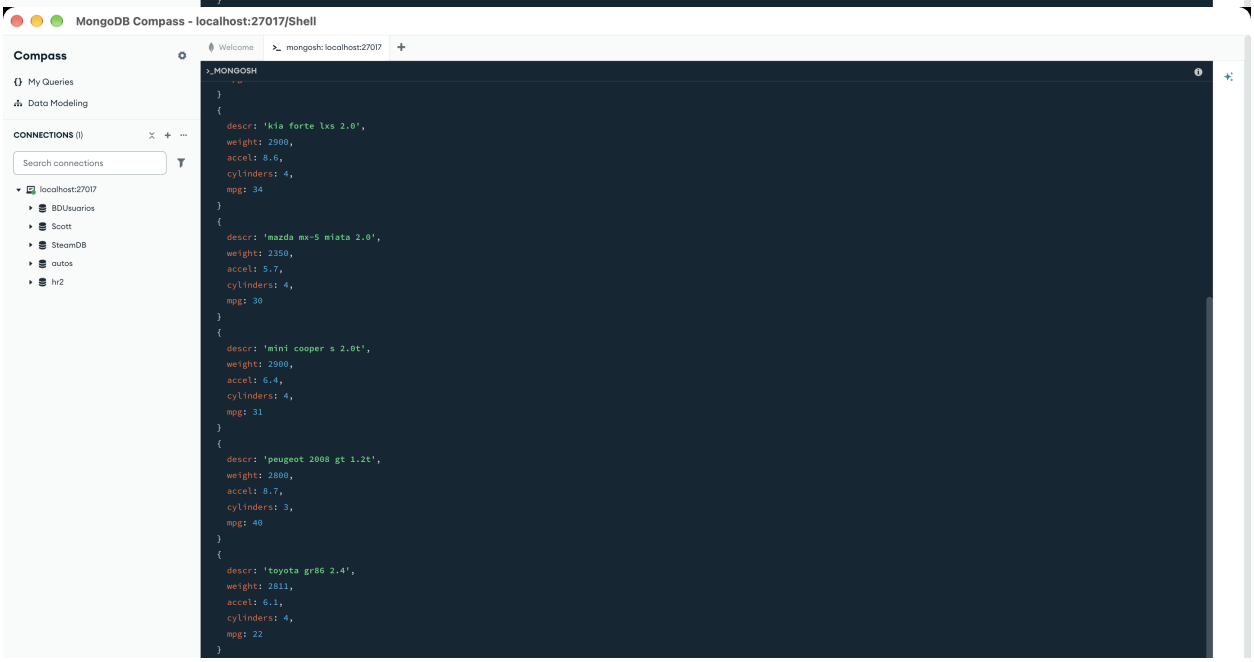
Método: CarsService.query3(), colección cars



4. Encontrar los autos que cumplan cualquiera de estas condiciones: (a) pesan menos de 3000 lb y aceleran de 0 a 60 mph en menos de 9 s, o (b) tienen 8 o más cilindros y rinden al menos 19 mpg.

Método: CarsService.query4(), colección cars

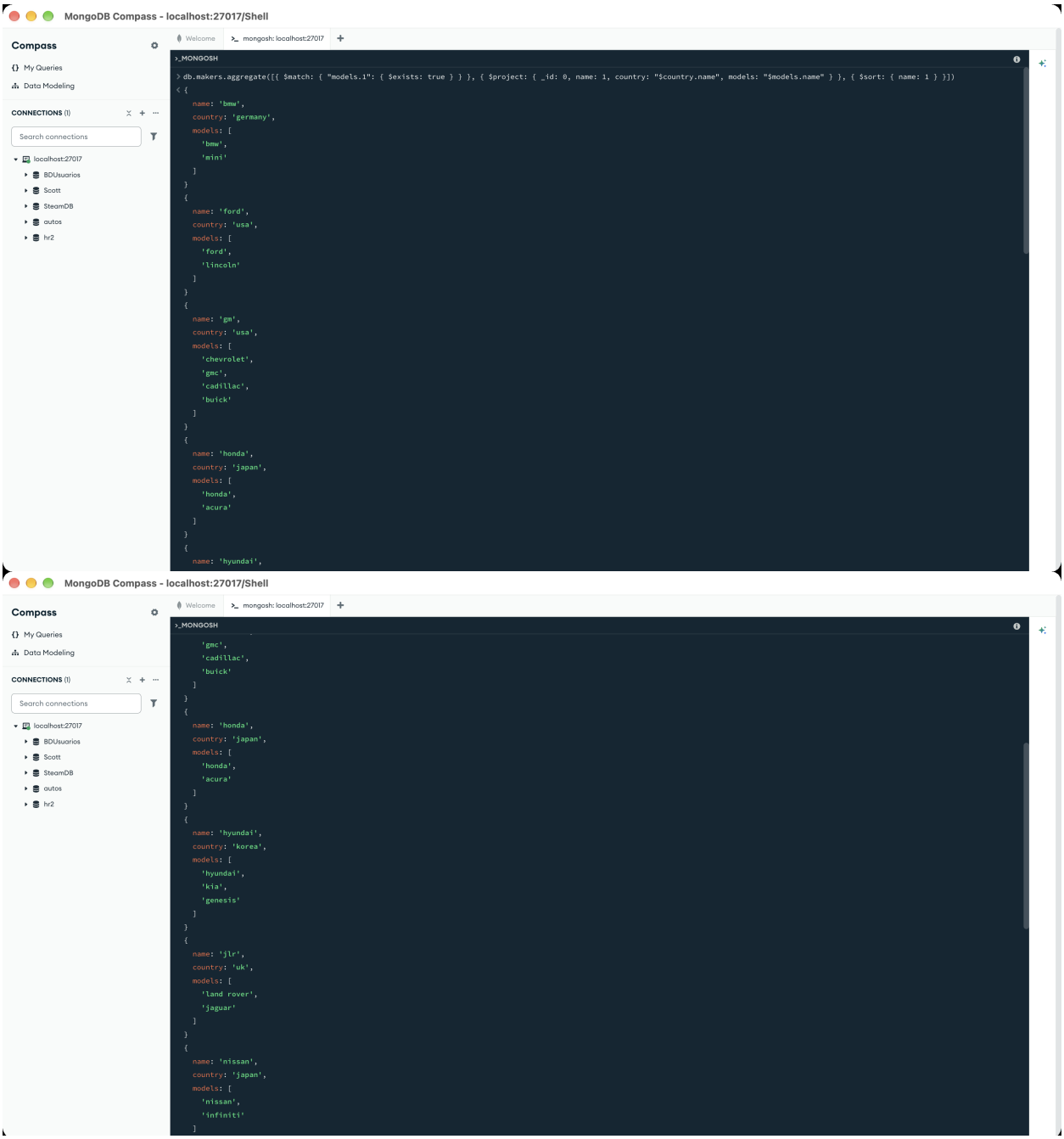


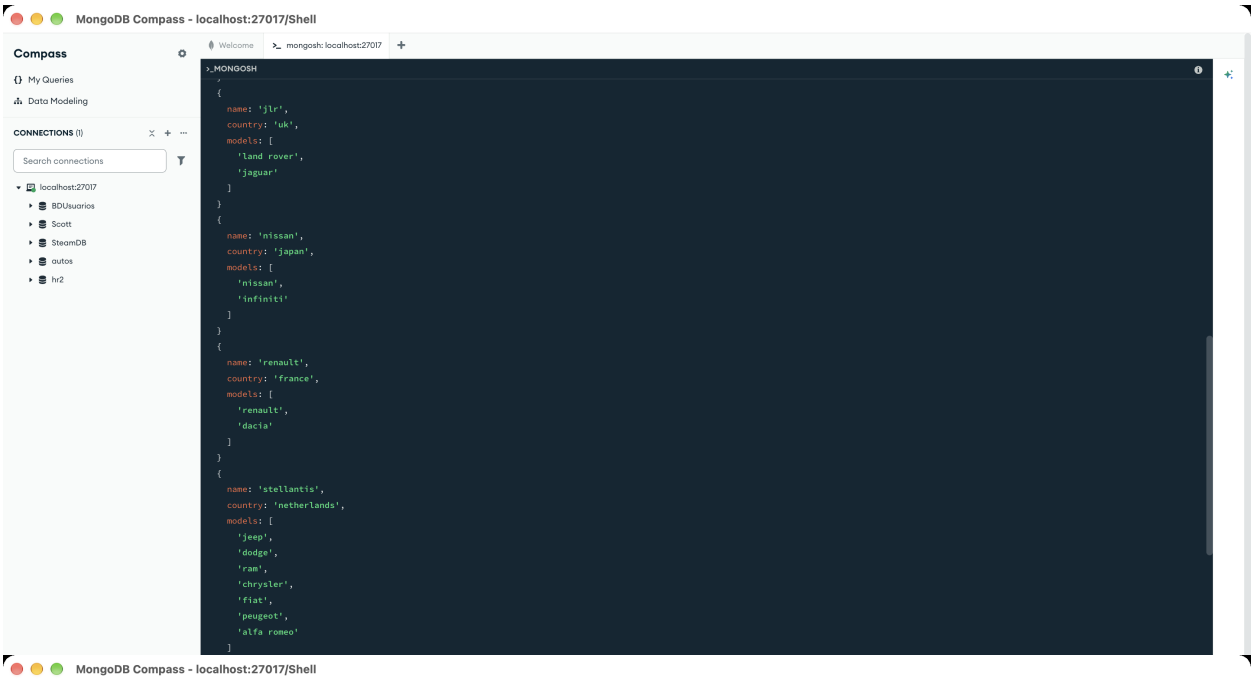


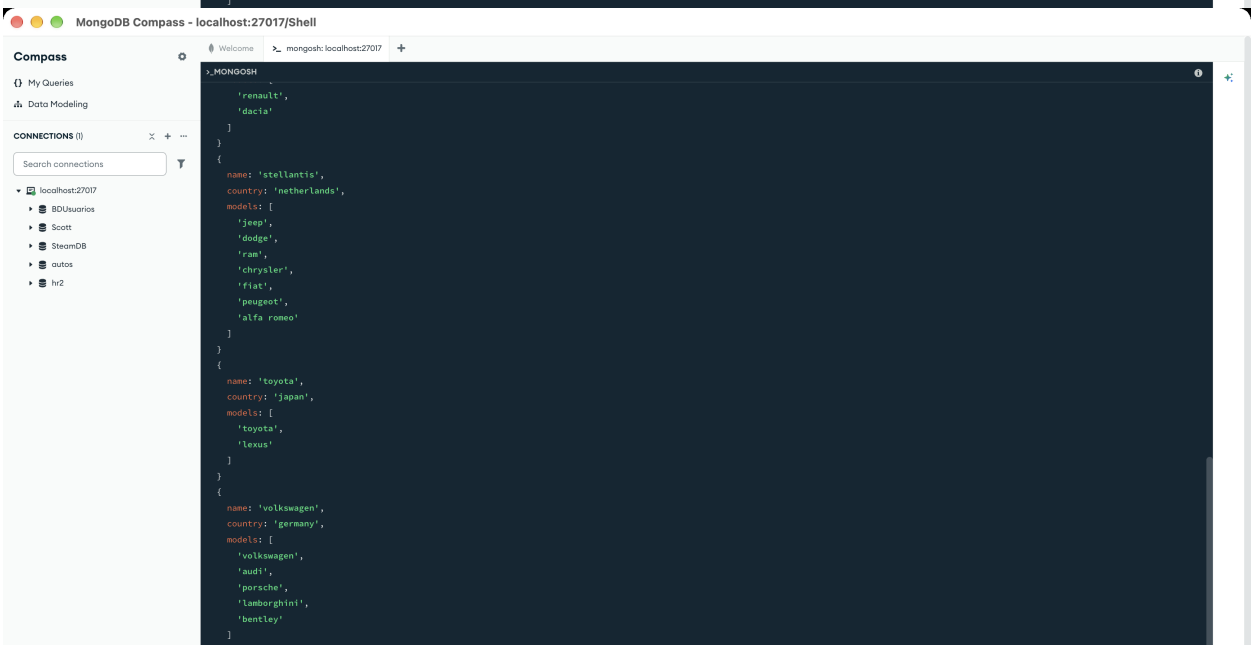
5. Obtener los fabricantes que comercializan más de una marca o línea de modelo, mostrando nombre, país y modelos.

Método: MakersService.query5(), colección makers

models.1 existe solo cuando el arreglo embebido tiene al menos dos marcas.



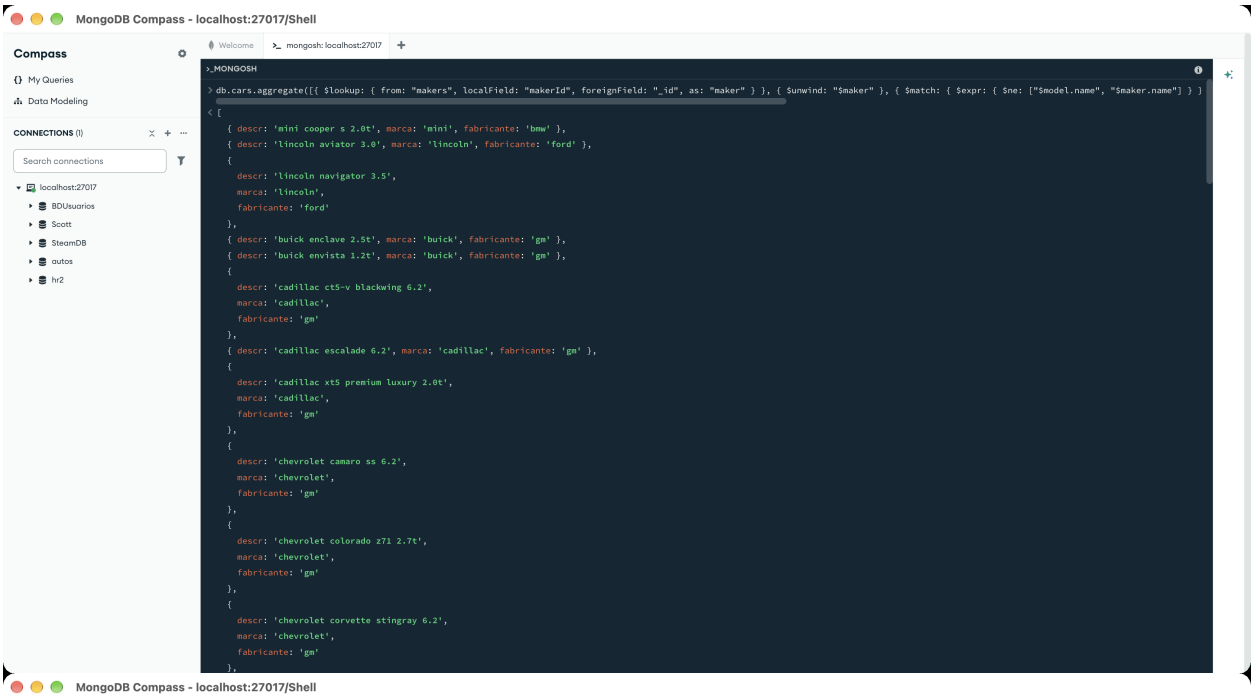

MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
{
name: 'jlr',
country: 'uk',
models: [
'land rover',
'jaguar'
]
}
{
name: 'nissan',
country: 'japan',
models: [
'nissan',
'infiniti'
]
}
{
name: 'renault',
country: 'france',
models: [
'renault',
'dacia'
]
}
{
name: 'stellantis',
country: 'netherlands',
models: [
'jeep',
'dodge',
'ram',
'chrysler',
'fiat',
'peugeot',
'alfa romeo'
]

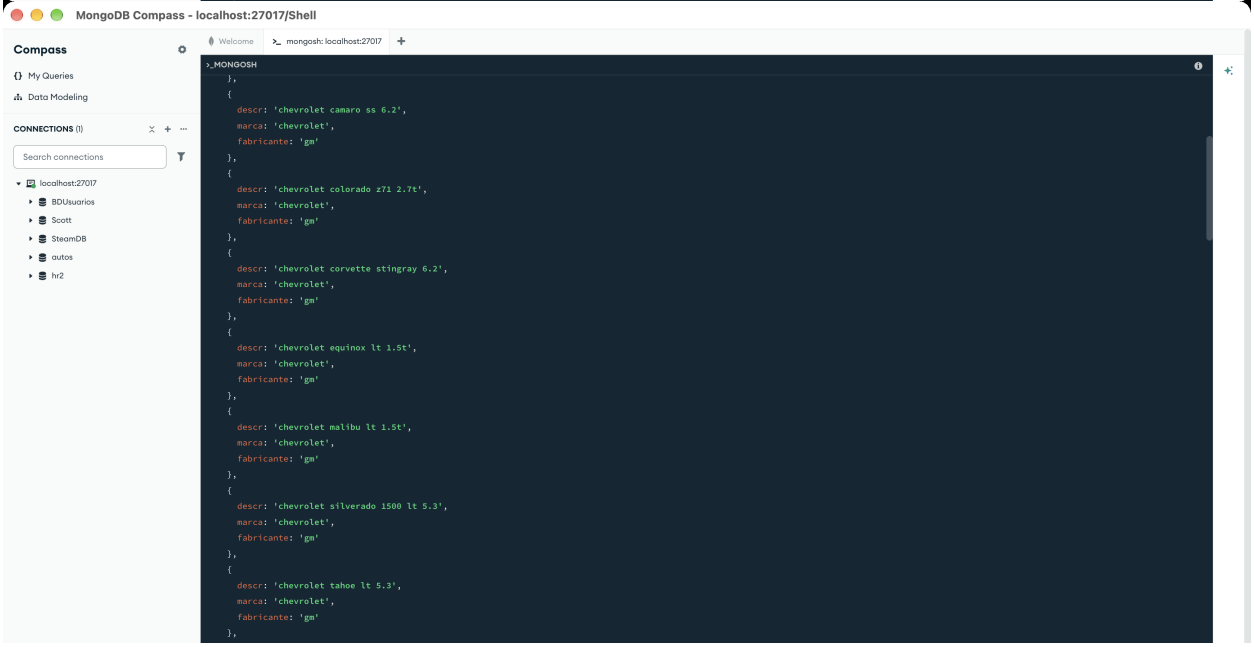

MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
'renault',
'dacia'
]
}
{
name: 'stellantis',
country: 'netherlands',
models: [
'jeep',
'dodge',
'ram',
'chrysler',
'fiat',
'peugeot',
'alfa romeo'
]
}
{
name: 'toyota',
country: 'japan',
models: [
'toyota',
'lexus'
]
}
{
name: 'volkswagen',
country: 'germany',
models: [
'volkswagen',
'audi',
'porsche',
'lamborghini',
'bentley'
]

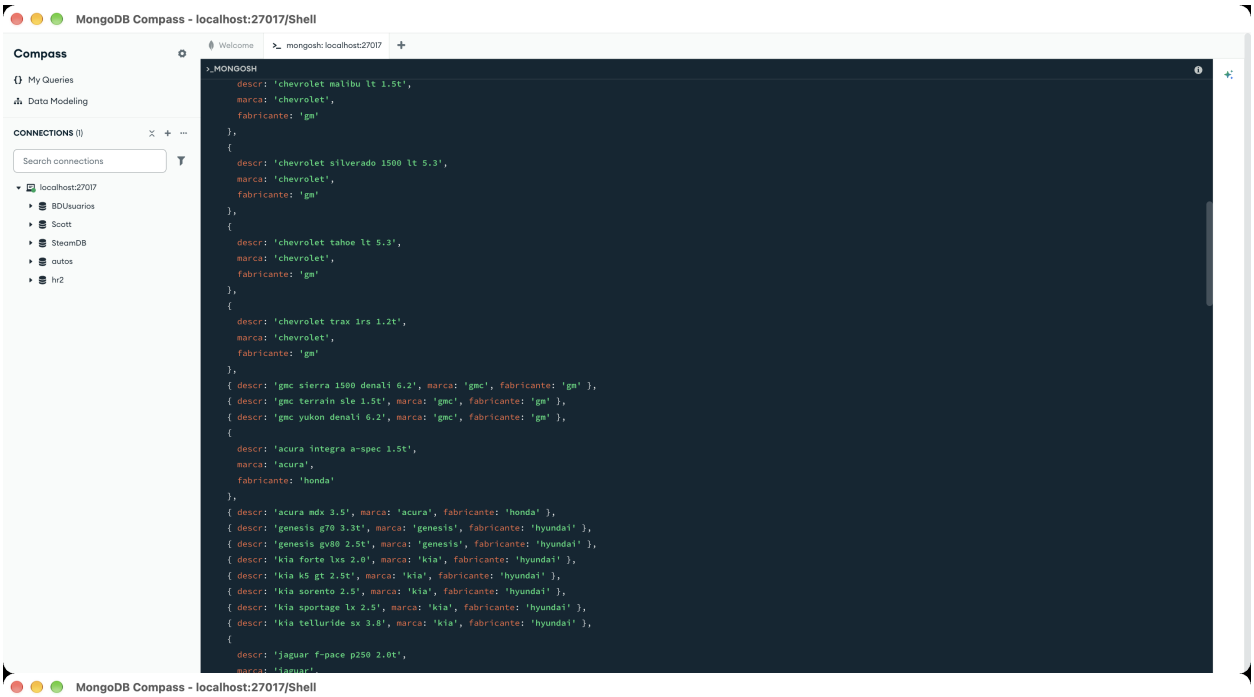
6. Encontrar los autos cuya marca (model.name) es distinta del nombre de su grupo fabricante (maker.name). Mostrar descripción, marca y fabricante.

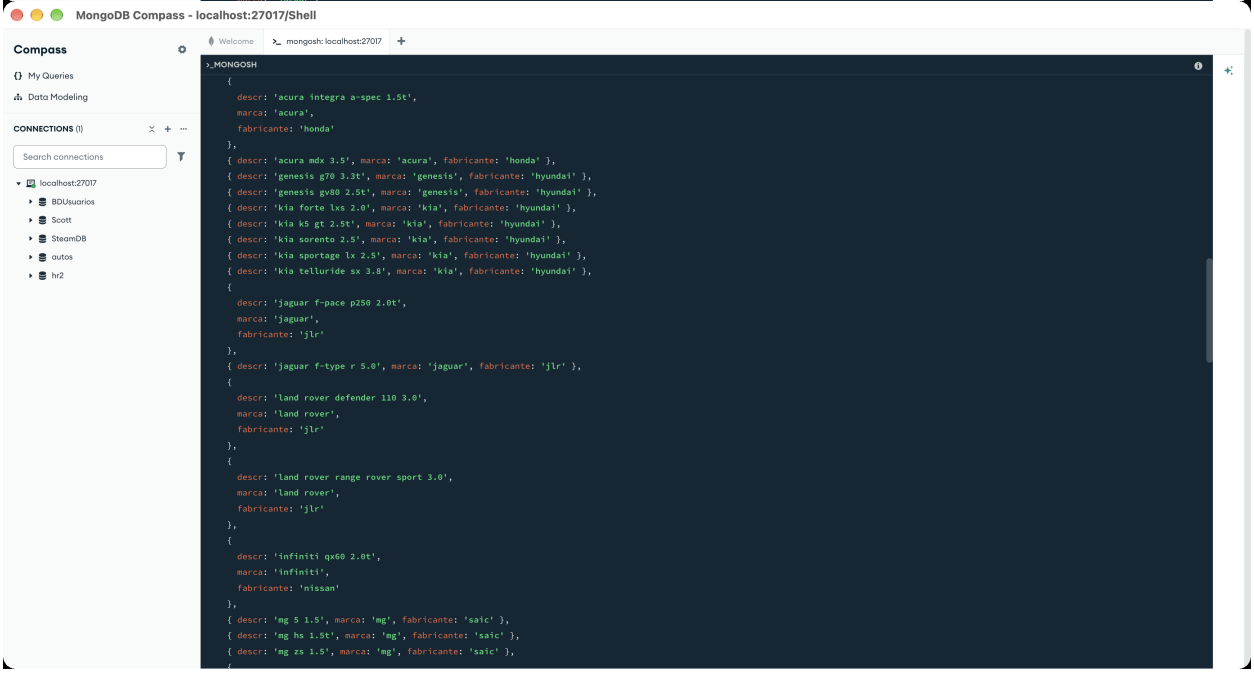
Método: CarsService.query6(), colección cars

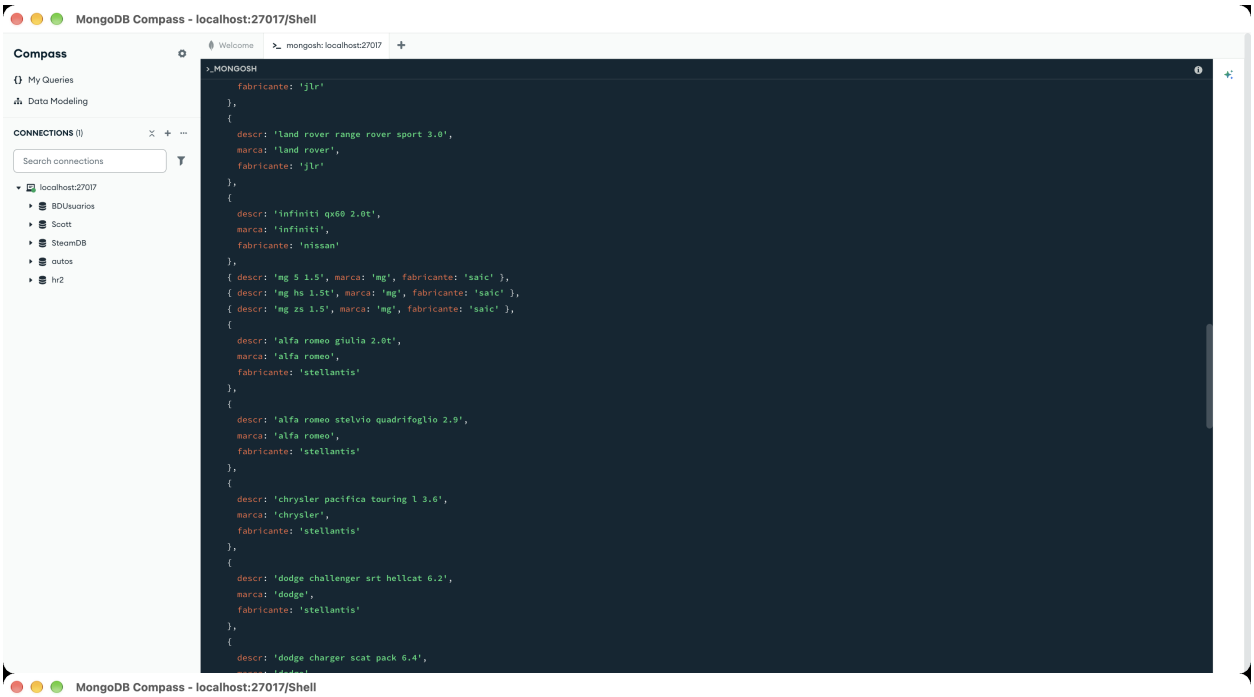
Regresa 60 documentos; le puse .toArray() para que el shell de Compass los mostrara todos.

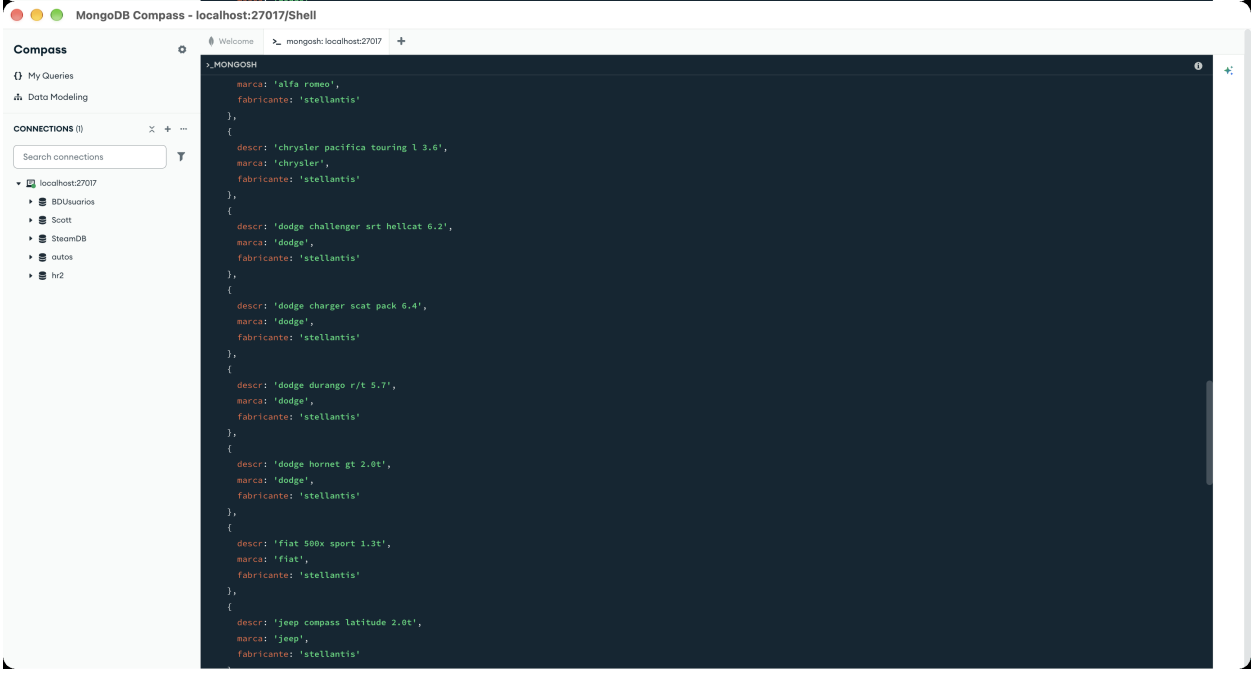


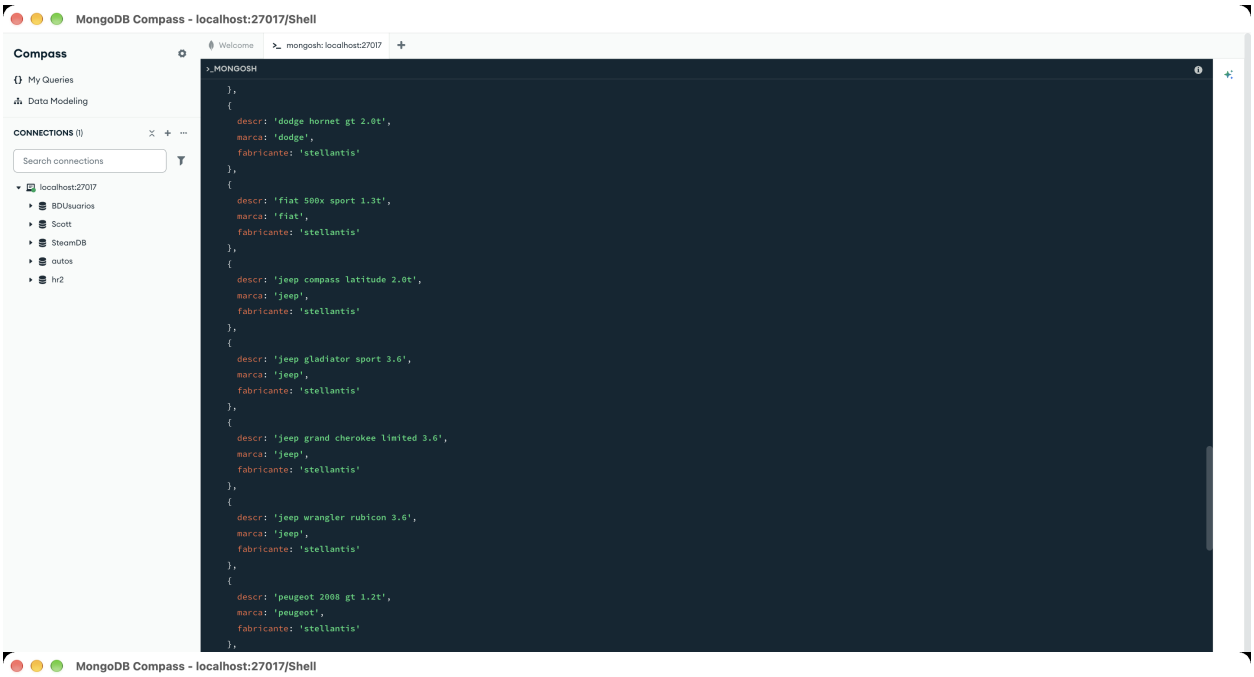


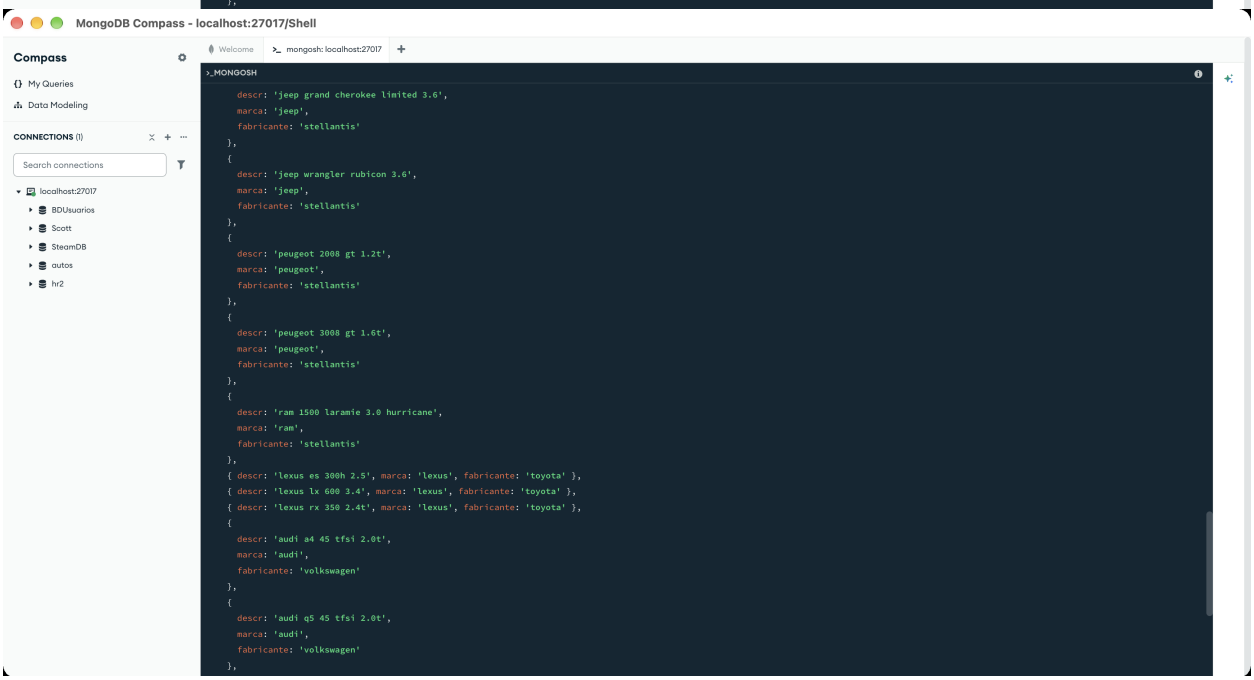

MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
descr: 'chevrolet malibu lt 1.5t',
marca: 'chevrolet',
fabricante: 'gm'
},
{
descr: 'chevrolet silverado 1500 lt 5.3',
marca: 'chevrolet',
fabricante: 'gm'
},
{
descr: 'chevrolet tahoe lt 5.3',
marca: 'chevrolet',
fabricante: 'gm'
},
{
descr: 'chevrolet trax 1rs 1.2t',
marca: 'chevrolet',
fabricante: 'gm'
},
{ descr: 'gmc sierra 1500 denali 6.2', marca: 'gmc', fabricante: 'gm' },
{ descr: 'gmc terrain sle 1.5t', marca: 'gmc', fabricante: 'gm' },
{ descr: 'gmc yukon denali 6.2', marca: 'gmc', fabricante: 'gm' },
{
descr: 'acura integra a-spec 1.5t',
marca: 'acura',
fabricante: 'honda'
},
{ descr: 'acura mdx 3.5', marca: 'acura', fabricante: 'honda' },
{ descr: 'genesis g70 3.3t', marca: 'genesis', fabricante: 'hyundai' },
{ descr: 'genesis gv80 2.5t', marca: 'genesis', fabricante: 'hyundai' },
{ descr: 'kia forte lxs 2.0', marca: 'kia', fabricante: 'hyundai' },
{ descr: 'kia k5 gt 2.5t', marca: 'kia', fabricante: 'hyundai' },
{ descr: 'kia sorento 2.5', marca: 'kia', fabricante: 'hyundai' },
{ descr: 'kia sportage lx 2.5', marca: 'kia', fabricante: 'hyundai' },
{ descr: 'kia telluride sx 3.8', marca: 'kia', fabricante: 'hyundai' },
{
descr: 'jaguar f-pace p250 2.0t',

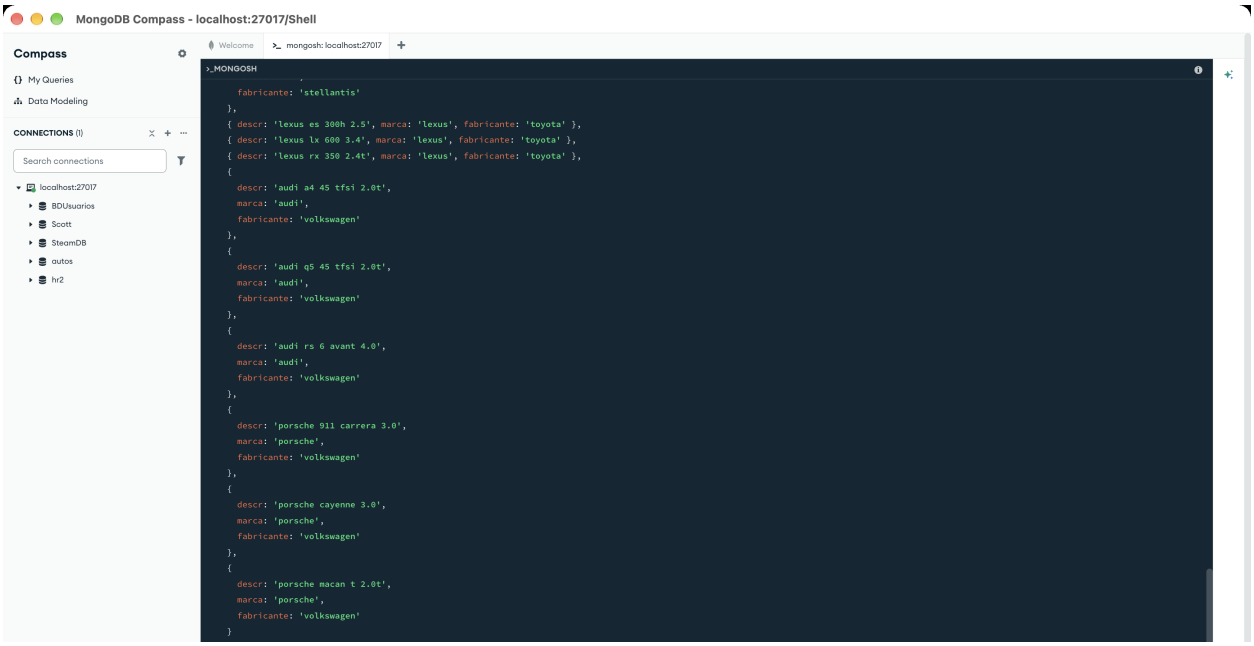

MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
{
descr: 'acura integra a-spec 1.5t',
marca: 'acura',
fabricante: 'honda'
},
{ descr: 'acura mdx 3.5', marca: 'acura', fabricante: 'honda' },
{ descr: 'genesis g70 3.3t', marca: 'genesis', fabricante: 'hyundai' },
{ descr: 'genesis gv80 2.5t', marca: 'genesis', fabricante: 'hyundai' },
{ descr: 'kia forte lxs 2.0', marca: 'kia', fabricante: 'hyundai' },
{ descr: 'kia k5 gt 2.5t', marca: 'kia', fabricante: 'hyundai' },
{ descr: 'kia sorento 2.5', marca: 'kia', fabricante: 'hyundai' },
{ descr: 'kia sportage lx 2.5', marca: 'kia', fabricante: 'hyundai' },
{ descr: 'kia telluride sx 3.8', marca: 'kia', fabricante: 'hyundai' },
{
descr: 'jaguar f-pace p250 2.0t',
marca: 'jaguar',
fabricante: 'jlr'
},
{ descr: 'jaguar f-type r 5.0', marca: 'jaguar', fabricante: 'jlr' },
{
descr: 'land rover defender 110 3.0',
marca: 'land rover',
fabricante: 'jlr'
},
{
descr: 'land rover range rover sport 3.0',
marca: 'land rover',
fabricante: 'jlr'
},
{
descr: 'infiniti qx60 2.0t',
marca: 'infiniti',
fabricante: 'nissan'
},
{ descr: 'mg 5 1.5', marca: 'mg', fabricante: 'saic' },
{ descr: 'mg hs 1.5t', marca: 'mg', fabricante: 'saic' },
{ descr: 'mg zs 1.5', marca: 'mg', fabricante: 'saic' },


MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
fabricante: 'jlr'
},
{
descr: 'land rover range rover sport 3.0',
marca: 'land rover',
fabricante: 'jlr'
},
{
descr: 'infiniti qx60 2.0t',
marca: 'infiniti',
fabricante: 'nissan'
},
{ descr: 'mg 5 1.5', marca: 'mg', fabricante: 'saic' },
{ descr: 'mg hs 1.5t', marca: 'mg', fabricante: 'saic' },
{ descr: 'mg zs 1.5', marca: 'mg', fabricante: 'saic' },
{
descr: 'alfa romeo giulia 2.0t',
marca: 'alfa romeo',
fabricante: 'stellantis'
},
{
descr: 'alfa romeo stelvio quadrifoglio 2.9',
marca: 'alfa romeo',
fabricante: 'stellantis'
},
{
descr: 'chrysler pacifica touring l 3.6',
marca: 'chrysler',
fabricante: 'stellantis'
},
{
descr: 'dodge challenger srt hellcat 6.2',
marca: 'dodge',
fabricante: 'stellantis'
},
{
descr: 'dodge charger scat pack 6.4',


MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
marca: 'alfa romeo',
fabricante: 'stellantis'
},
{
descr: 'chrysler pacifica touring l 3.6',
marca: 'chrysler',
fabricante: 'stellantis'
},
{
descr: 'dodge challenger srt hellcat 6.2',
marca: 'dodge',
fabricante: 'stellantis'
},
{
descr: 'dodge charger scat pack 6.4',
marca: 'dodge',
fabricante: 'stellantis'
},
{
descr: 'dodge durango r/t 5.7',
marca: 'dodge',
fabricante: 'stellantis'
},
{
descr: 'dodge hornet gt 2.0t',
marca: 'dodge',
fabricante: 'stellantis'
},
{
descr: 'fiat 500x sport 1.3t',
marca: 'fiat',
fabricante: 'stellantis'
},
{
descr: 'jeep compass latitude 2.0t',
marca: 'jeep',
fabricante: 'stellantis'

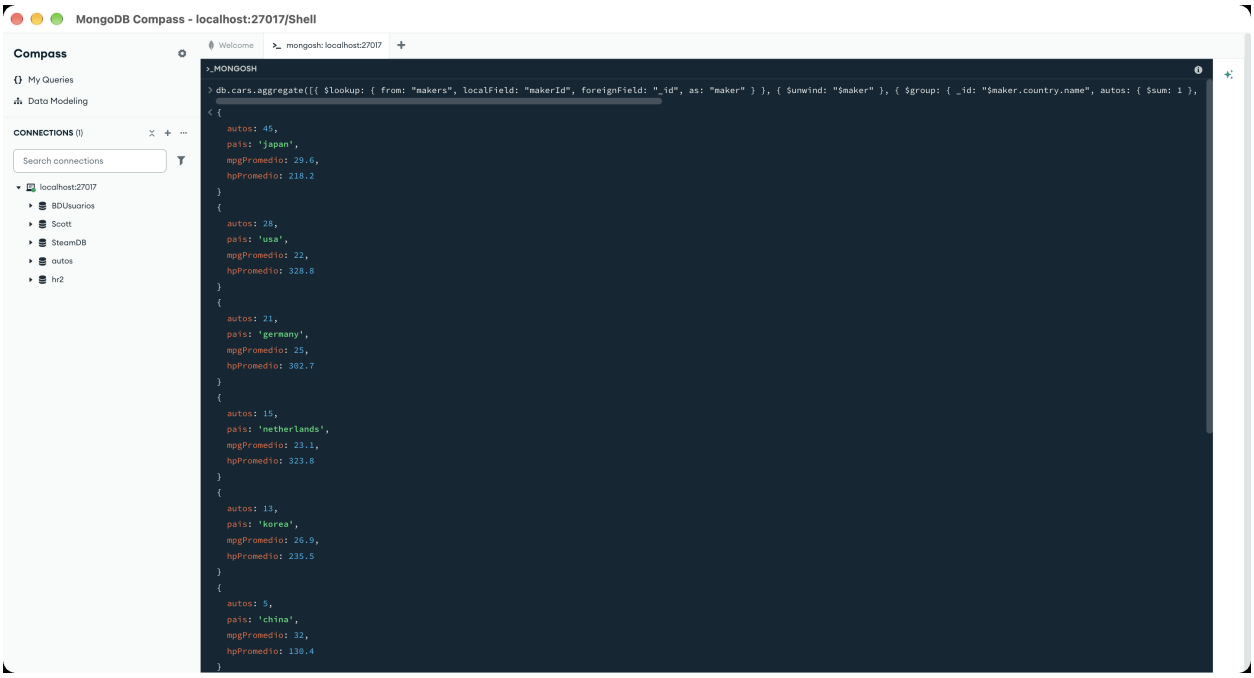

MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
},
{
descr: 'dodge hornet gt 2.0t',
marca: 'dodge',
fabricante: 'stellantis'
},
{
descr: 'fiat 500x sport 1.3t',
marca: 'fiat',
fabricante: 'stellantis'
},
{
descr: 'jeep compass latitude 2.0t',
marca: 'jeep',
fabricante: 'stellantis'
},
{
descr: 'jeep gladiator sport 3.6',
marca: 'jeep',
fabricante: 'stellantis'
},
{
descr: 'jeep grand cherokee limited 3.6',
marca: 'jeep',
fabricante: 'stellantis'
},
{
descr: 'jeep wrangler rubicon 3.6',
marca: 'jeep',
fabricante: 'stellantis'
},
{
descr: 'peugeot 2008 gt 1.2t',
marca: 'peugeot',
fabricante: 'stellantis'
},

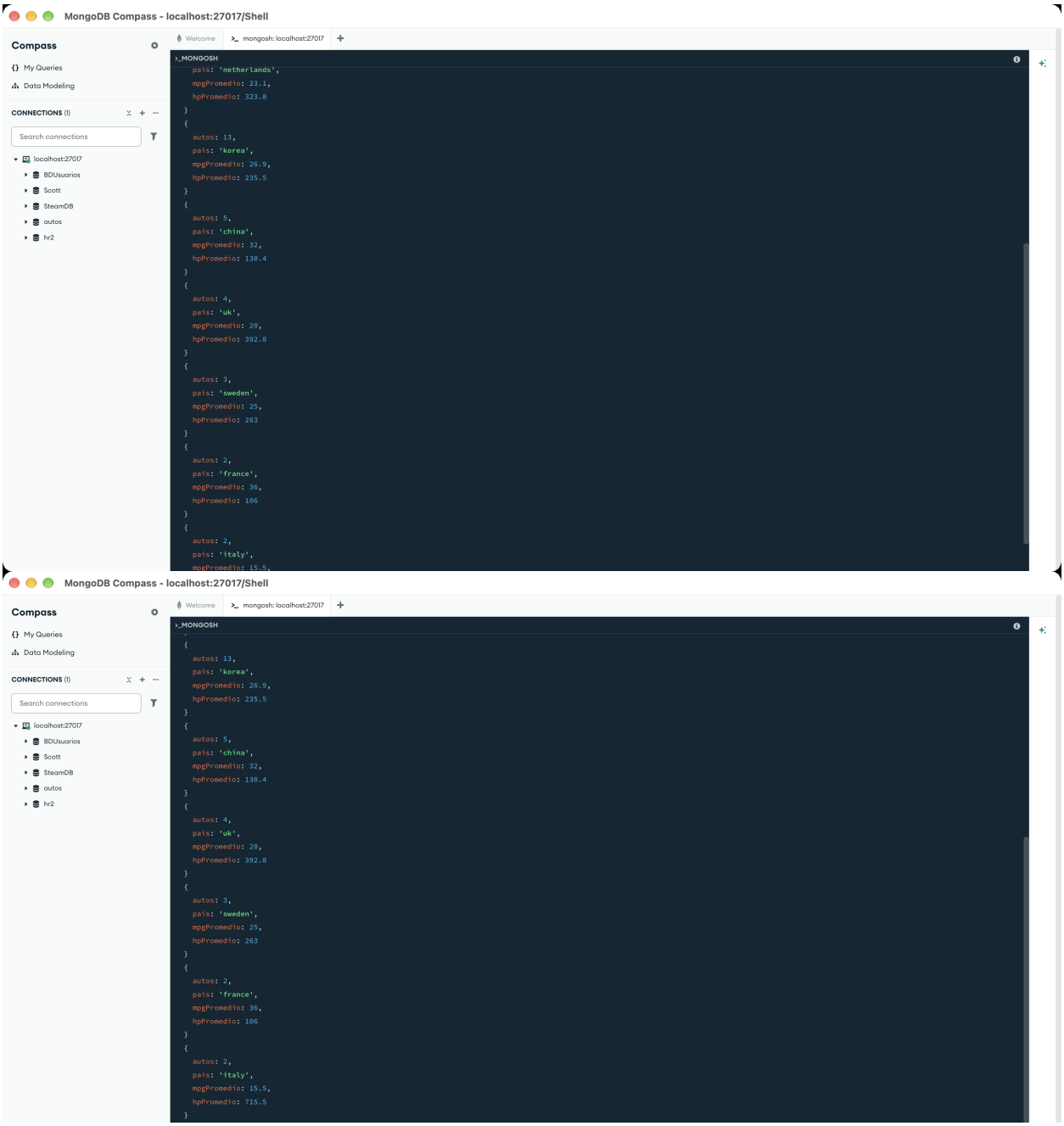

MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
descr: 'jeep grand cherokee limited 3.6',
marca: 'jeep',
fabricante: 'stellantis'
},
{
descr: 'jeep wrangler rubicon 3.6',
marca: 'jeep',
fabricante: 'stellantis'
},
{
descr: 'peugeot 2008 gt 1.2t',
marca: 'peugeot',
fabricante: 'stellantis'
},
{
descr: 'peugeot 3008 gt 1.6t',
marca: 'peugeot',
fabricante: 'stellantis'
},
{
descr: 'ram 1500 laramie 3.0 hurricane',
marca: 'ram',
fabricante: 'stellantis'
},
{ descr: 'lexus es 300h 2.5', marca: 'lexus', fabricante: 'toyota' },
{ descr: 'lexus lx 600 3.4', marca: 'lexus', fabricante: 'toyota' },
{ descr: 'lexus rx 350 2.4t', marca: 'lexus', fabricante: 'toyota' },
{
descr: 'audi a4 45 tfsi 2.0t',
marca: 'audi',
fabricante: 'volkswagen'
},
{
descr: 'audi q5 45 tfsi 2.0t',
marca: 'audi',
fabricante: 'volkswagen'
},



7. Calcular por país el número de autos, el rendimiento promedio y la potencia promedio (redondeados a 1 decimal), ordenado por número de autos.

Método: CarsService.query7(), colección cars

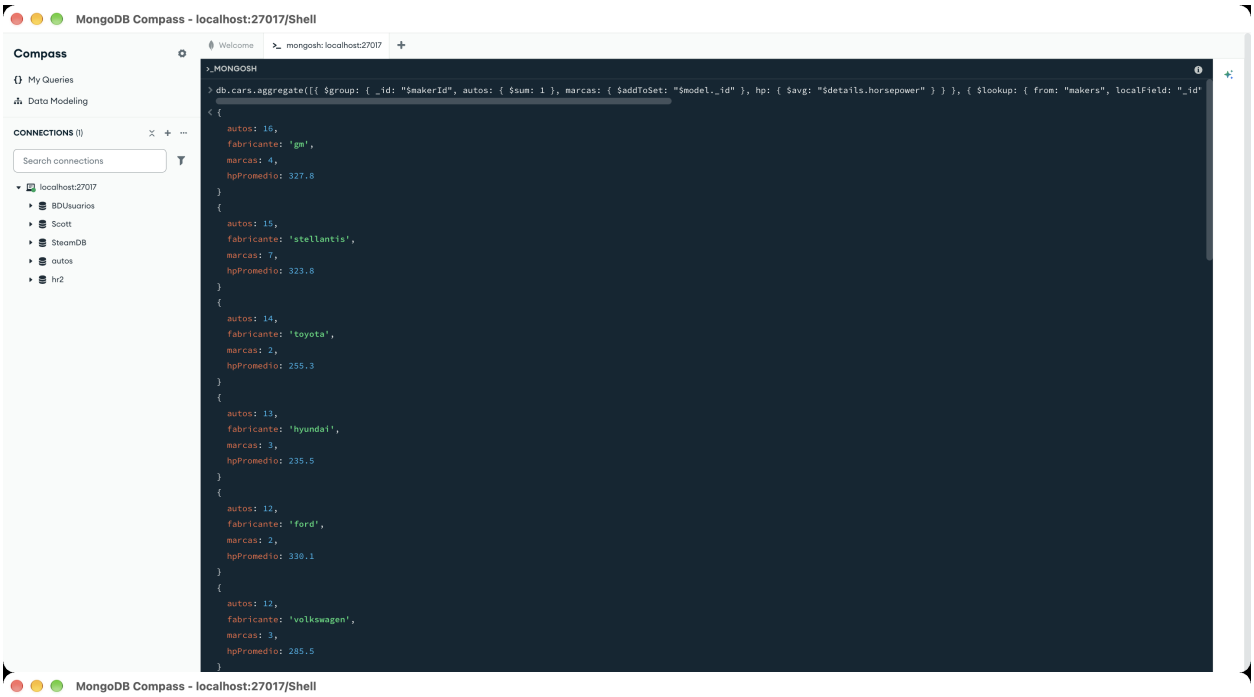


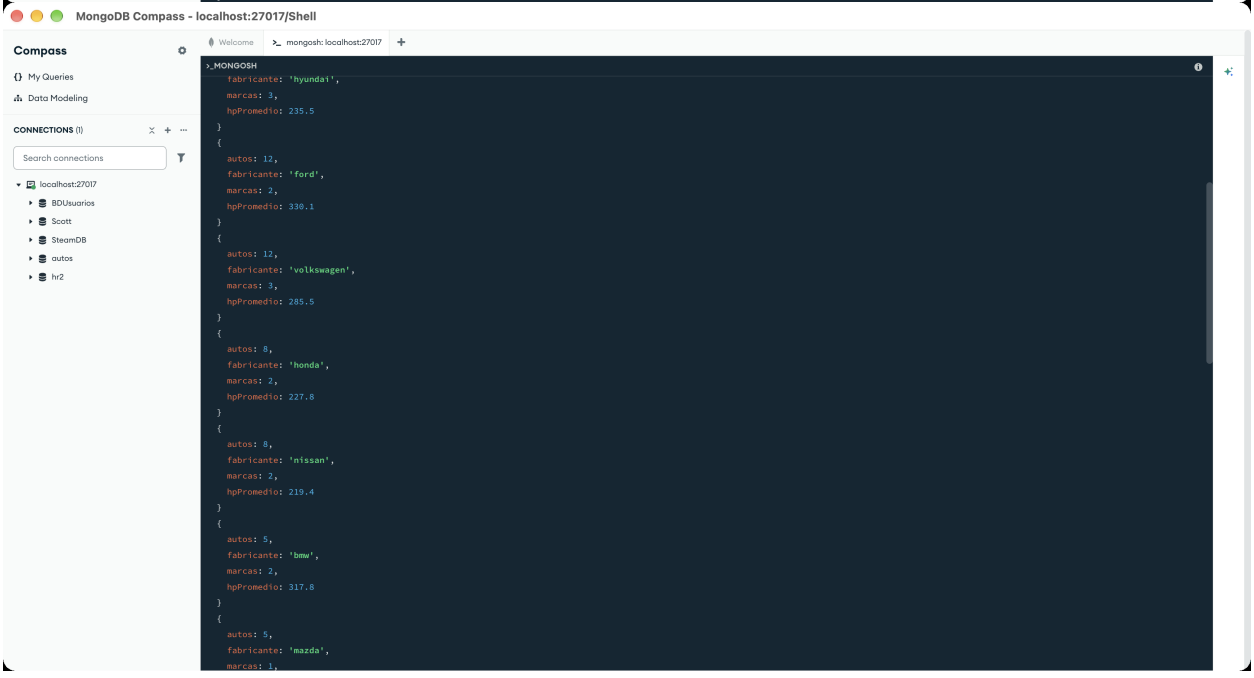

MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
pais: 'netherlands',
mpgPromedio: 23.1,
hpPromedio: 323.8
}
{
autos: 13,
pais: 'korea',
mpgPromedio: 26.9,
hpPromedio: 235.5
}
{
autos: 5,
pais: 'china',
mpgPromedio: 32,
hpPromedio: 130.4
}
{
autos: 4,
pais: 'uk',
mpgPromedio: 20,
hpPromedio: 392.8
}
{
autos: 3,
pais: 'sweden',
mpgPromedio: 25,
hpPromedio: 263
}
{
autos: 2,
pais: 'france',
mpgPromedio: 36,
hpPromedio: 106
}
{
autos: 2,
pais: 'italy',
mpgPromedio: 15.5,

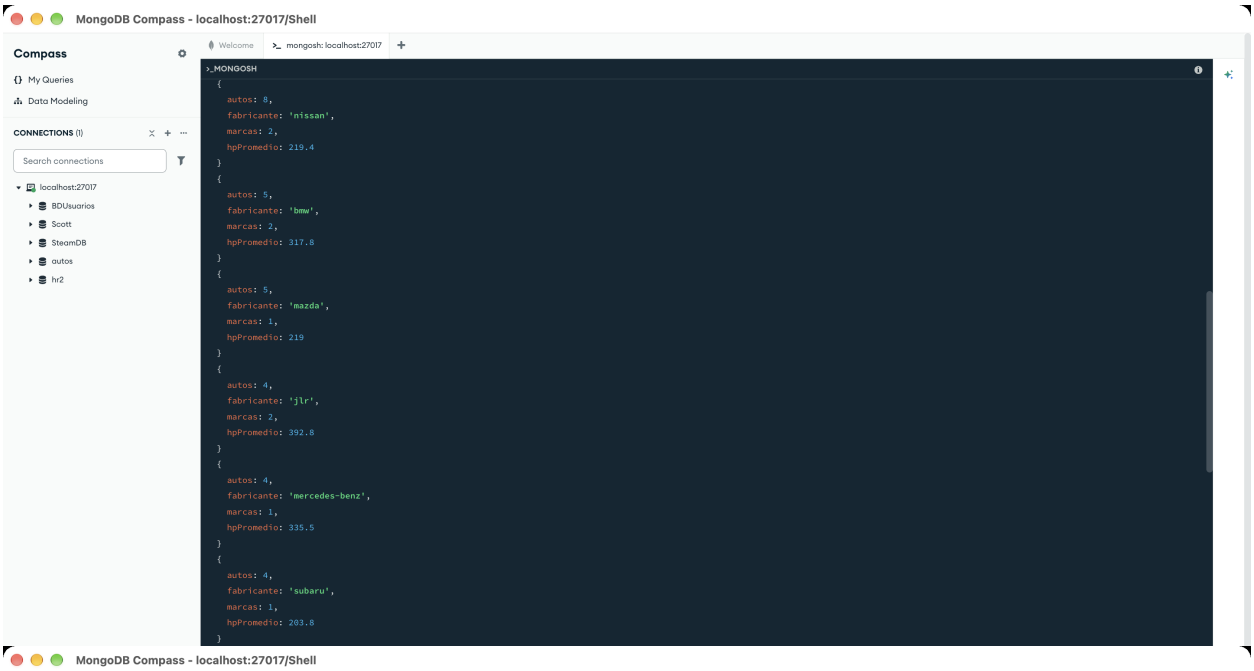
MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
{
autos: 13,
pais: 'korea',
mpgPromedio: 26.9,
hpPromedio: 235.5
}
{
autos: 5,
pais: 'china',
mpgPromedio: 32,
hpPromedio: 130.4
}
{
autos: 4,
pais: 'uk',
mpgPromedio: 20,
hpPromedio: 392.8
}
{
autos: 3,
pais: 'sweden',
mpgPromedio: 25,
hpPromedio: 263
}
{
autos: 2,
pais: 'france',
mpgPromedio: 36,
hpPromedio: 106
}
{
autos: 2,
pais: 'italy',
mpgPromedio: 15.5,
hpPromedio: 715.5
}

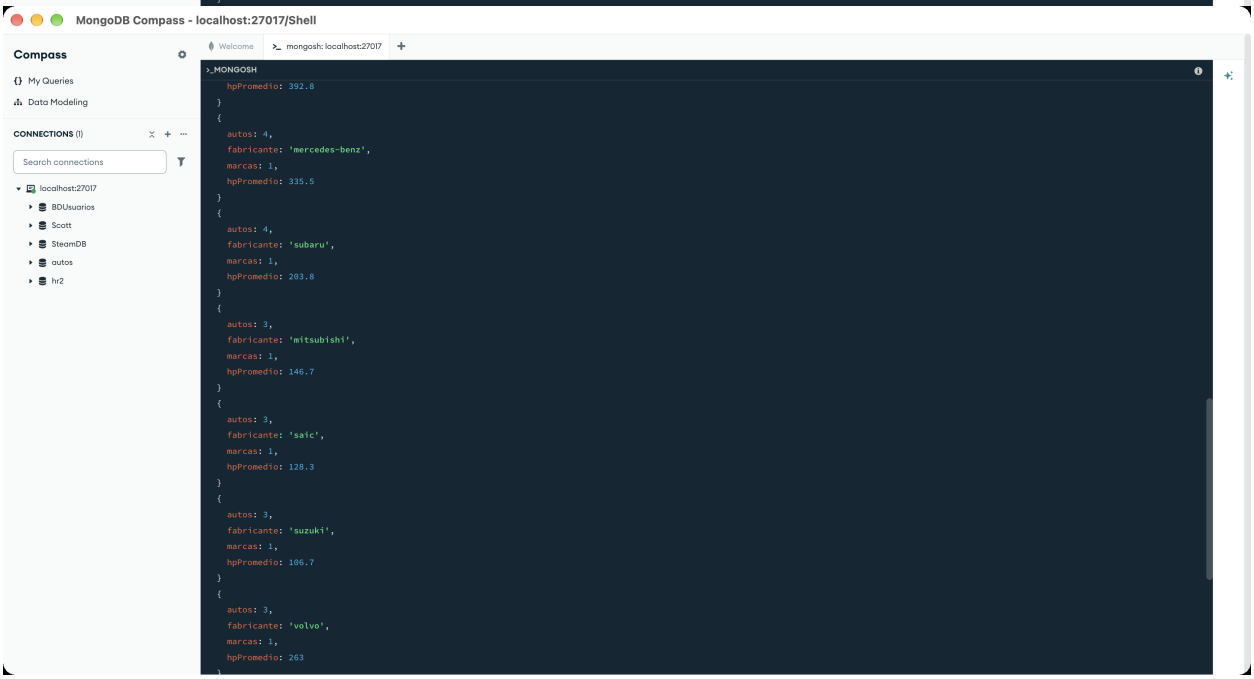
8. Calcular para cada grupo fabricante el número de autos, el número de marcas distintas con autos registrados y la potencia promedio (redondeada a 1 decimal), ordenado por número de autos.

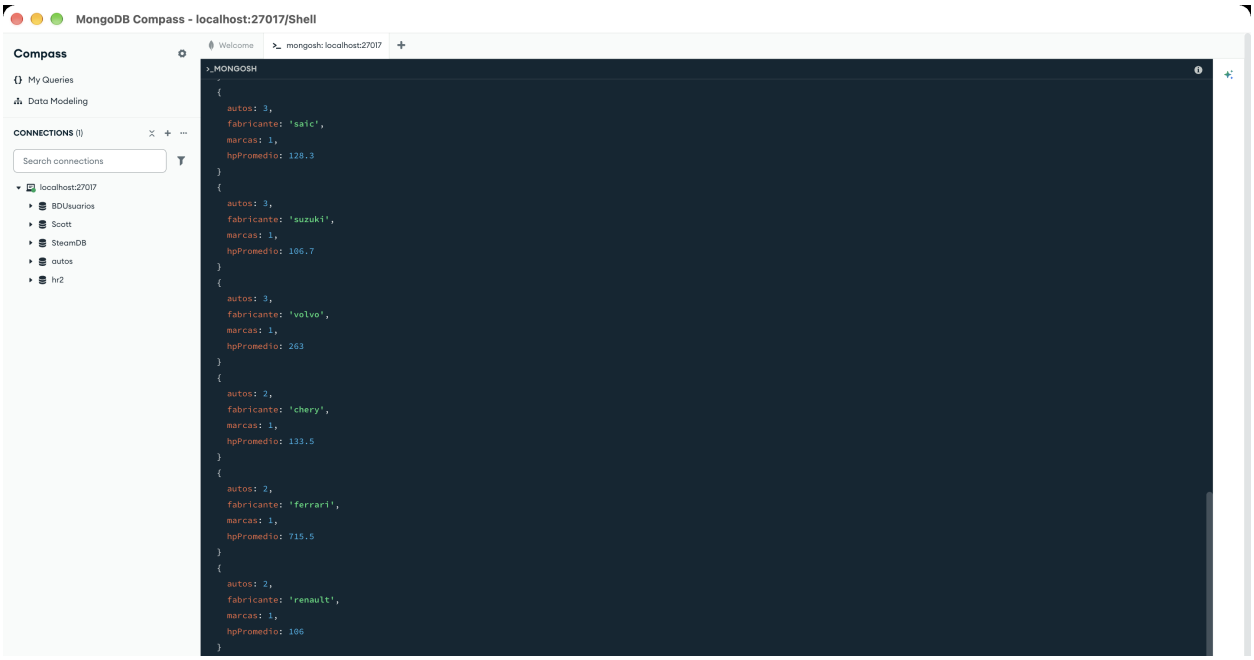
Método: CarsService.query8(), colección cars






MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
{
autos: 8,
fabricante: 'nissan',
marcas: 2,
hpPromedio: 219.4
}
{
autos: 5,
fabricante: 'bmw',
marcas: 2,
hpPromedio: 317.8
}
{
autos: 5,
fabricante: 'mazda',
marcas: 1,
hpPromedio: 219
}
{
autos: 4,
fabricante: 'jlr',
marcas: 2,
hpPromedio: 392.8
}
{
autos: 4,
fabricante: 'mercedes-benz',
marcas: 1,
hpPromedio: 335.5
}
{
autos: 4,
fabricante: 'subaru',
marcas: 1,
hpPromedio: 203.8
}

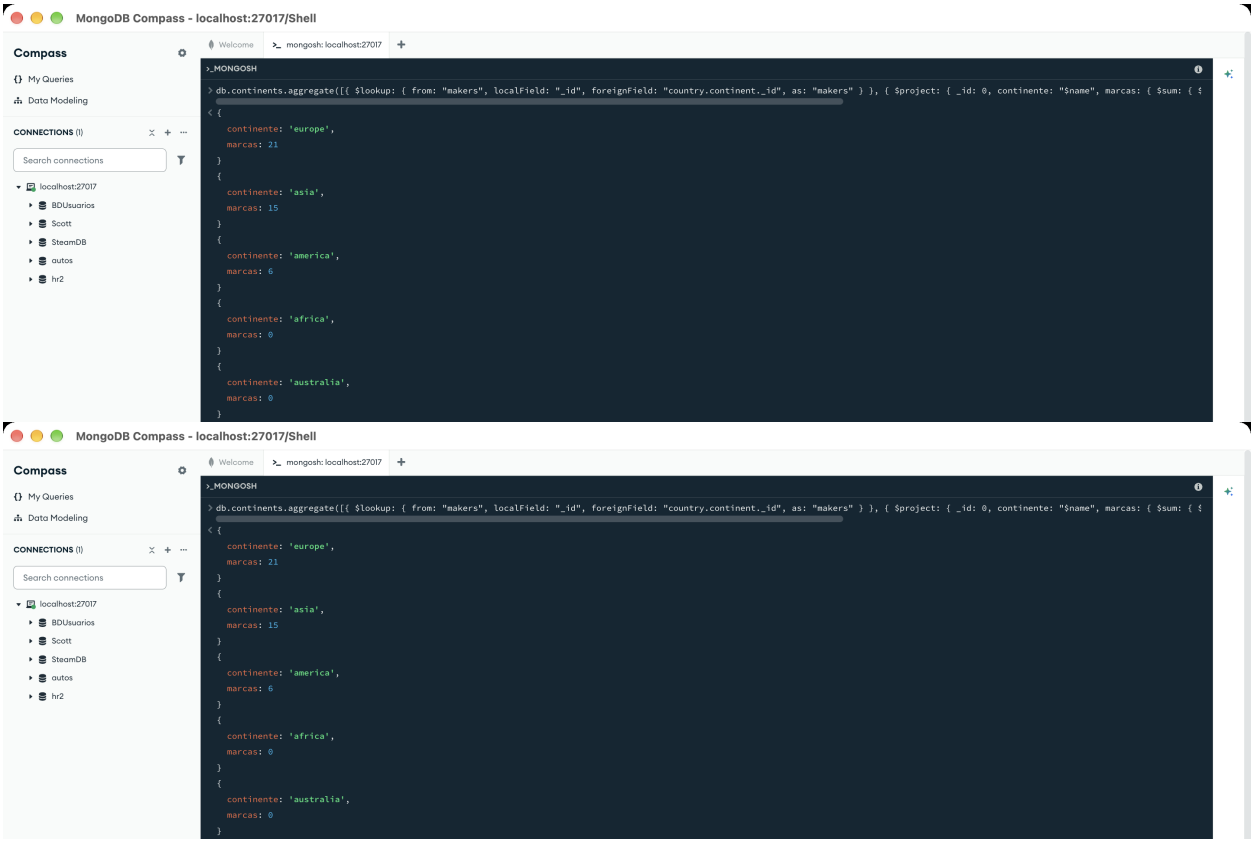

MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
hpPromedio: 392.8
}
{
autos: 4,
fabricante: 'mercedes-benz',
marcas: 1,
hpPromedio: 335.5
}
{
autos: 4,
fabricante: 'subaru',
marcas: 1,
hpPromedio: 203.8
}
{
autos: 3,
fabricante: 'mitsubishi',
marcas: 1,
hpPromedio: 146.7
}
{
autos: 3,
fabricante: 'saic',
marcas: 1,
hpPromedio: 128.3
}
{
autos: 3,
fabricante: 'suzuki',
marcas: 1,
hpPromedio: 106.7
}
{
autos: 3,
fabricante: 'volvo',
marcas: 1,
hpPromedio: 263
}



9. Contar cuántos modelos (marcas) hay por continente.

Método: ContinentsService.query9(), colección continents

Parte de continents para que aparezcan también los continentes sin marcas.

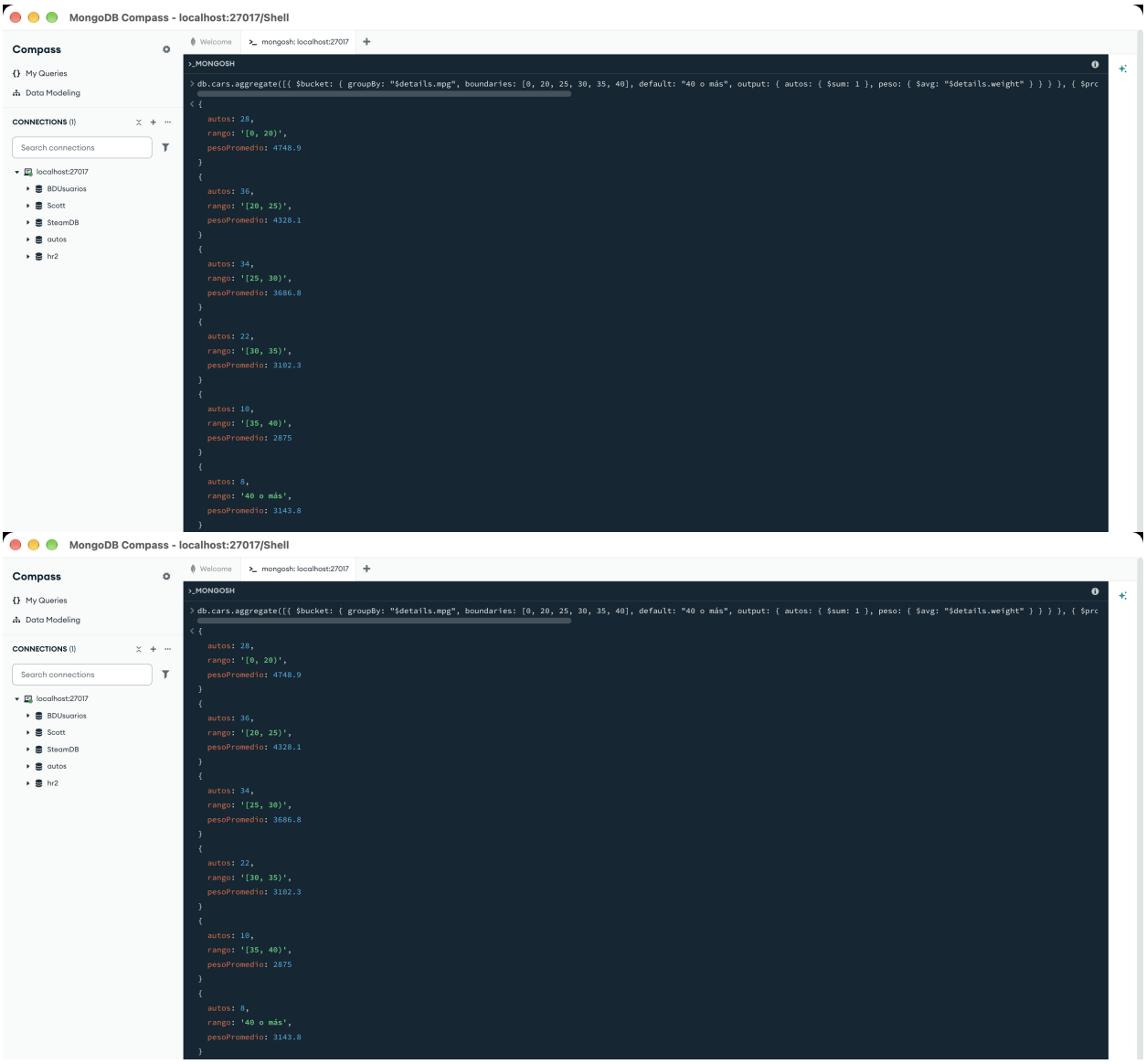


MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
db.continents.aggregate([{ $lookup: { from: "makers", localField: "_id", foreignField: "country.continent._id", as: "makers" } }, { $project: { _id: 0, continente: "$name", marcas: { $sum: { $
{
continente: 'europe',
marcas: 21
}
{
continente: 'asia',
marcas: 15
}
{
continente: 'america',
marcas: 6
}
{
continente: 'africa',
marcas: 0
}
{
continente: 'australia',
marcas: 0
}

10. Clasificar los autos en rangos de rendimiento [0,20), [20,25), [25,30), [30,35), [35,40) y 40 o más, contando cuántos hay en cada rango y su peso promedio.

Método: CarsService.query10(), colección cars

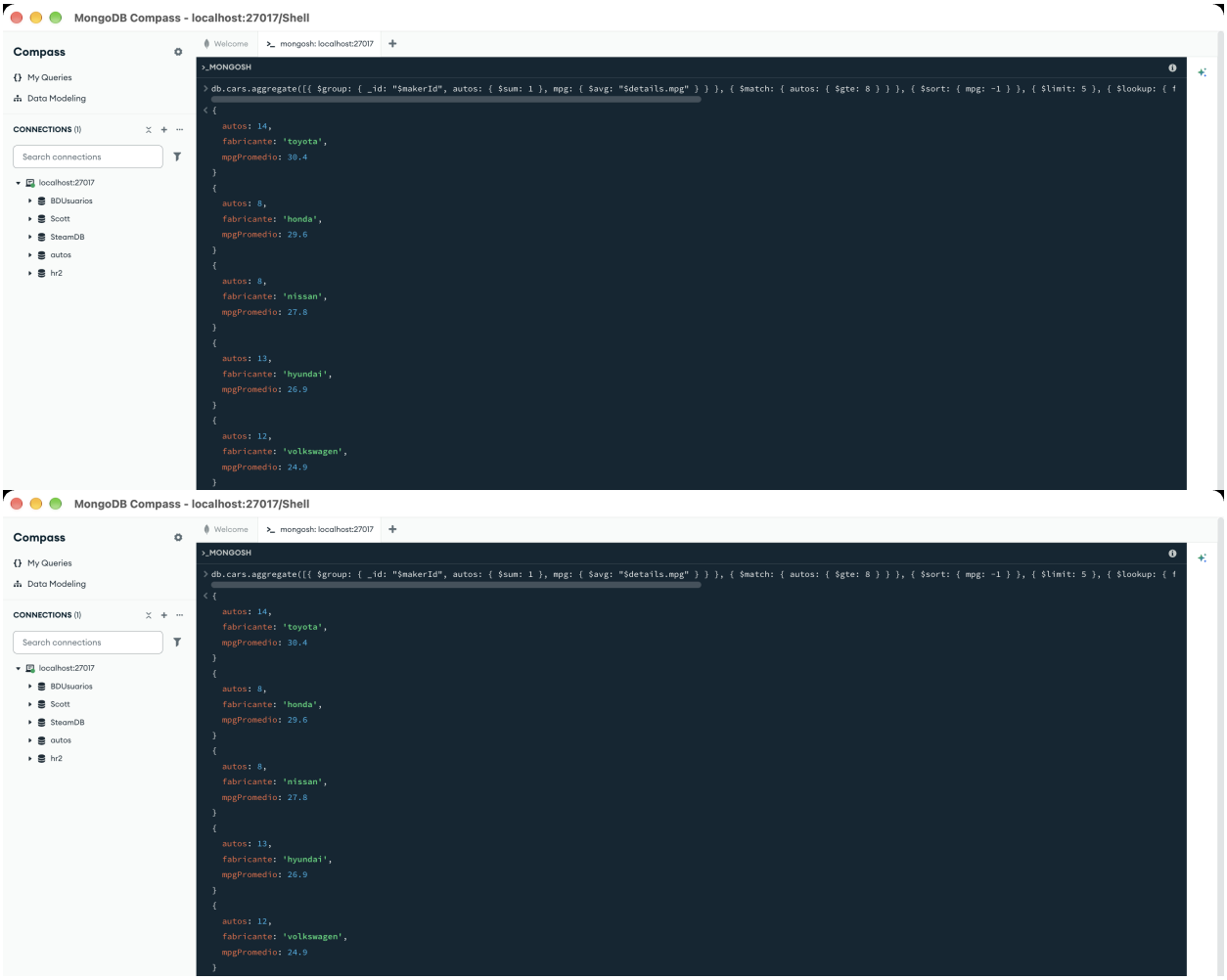
$bucket usa cada límite como inicio de un rango; lo que pasa de 40 cae en default.



MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
db.cars.aggregate([{ $bucket: { groupBy: "$details.mpg", boundaries: [0, 20, 25, 30, 35, 40], default: "40 o más", output: { autos: { $sum: 1 }, peso: { $avg: "$details.weight" } } } }, { $prc
{
autos: 28,
rango: '[0, 20)',
pesoPromedio: 4748.9
}
{
autos: 36,
rango: '[20, 25)',
pesoPromedio: 4328.1
}
{
autos: 34,
rango: '[25, 30)',
pesoPromedio: 3686.8
}
{
autos: 22,
rango: '[30, 35)',
pesoPromedio: 3102.3
}
{
autos: 10,
rango: '[35, 40)',
pesoPromedio: 2875
}
{
autos: 8,
rango: '40 o más',
pesoPromedio: 3143.8
}

11. Obtener los 5 fabricantes con mayor rendimiento promedio, considerando solo aquellos con al menos 8 autos registrados.

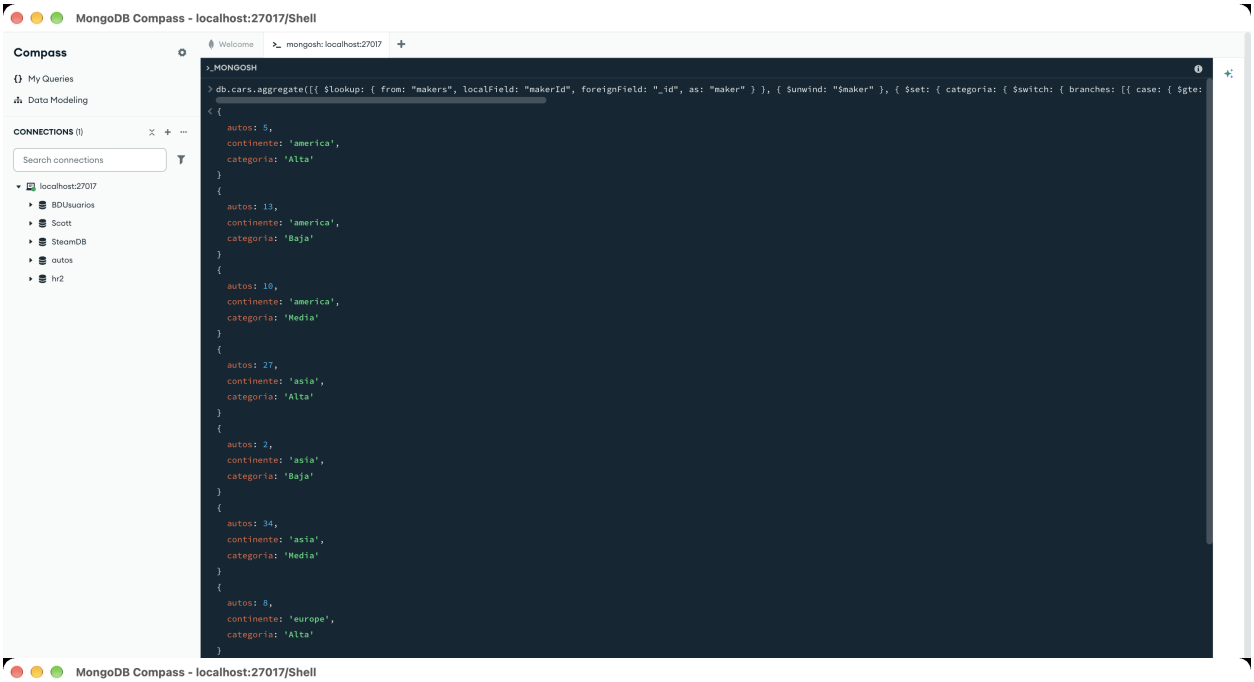
Método: CarsService.query11(), colección cars

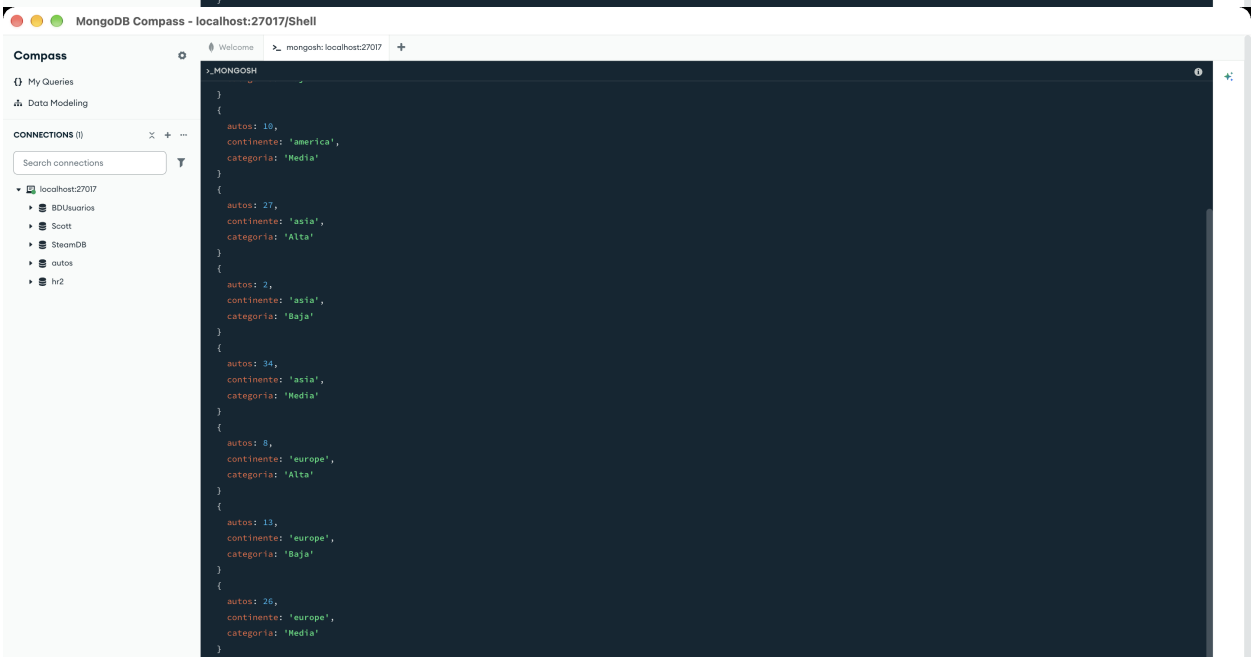


MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
db.cars.aggregate([{ $group: { _id: "$makerId", autos: { $sum: 1 }, mpg: { $avg: "$details.mpg" } } }, { $match: { autos: { $gte: 8 } } }, { $sort: { mpg: -1 } }, { $limit: 5 }, { $lookup: { f
{
autos: 14,
fabricante: 'toyota',
mpgPromedio: 30.4
}
{
autos: 8,
fabricante: 'honda',
mpgPromedio: 29.6
}
{
autos: 8,
fabricante: 'nissan',
mpgPromedio: 27.8
}
{
autos: 13,
fabricante: 'hyundai',
mpgPromedio: 26.9
}
{
autos: 12,
fabricante: 'volkswagen',
mpgPromedio: 24.9
}

12. Clasificar cada auto como "Alta" (mpg ≥ 30), "Media" (20 ≤ mpg < 30) o "Baja" (mpg < 20) y contar cuántos autos hay por continente y categoría.

Método: CarsService.query12(), colección cars

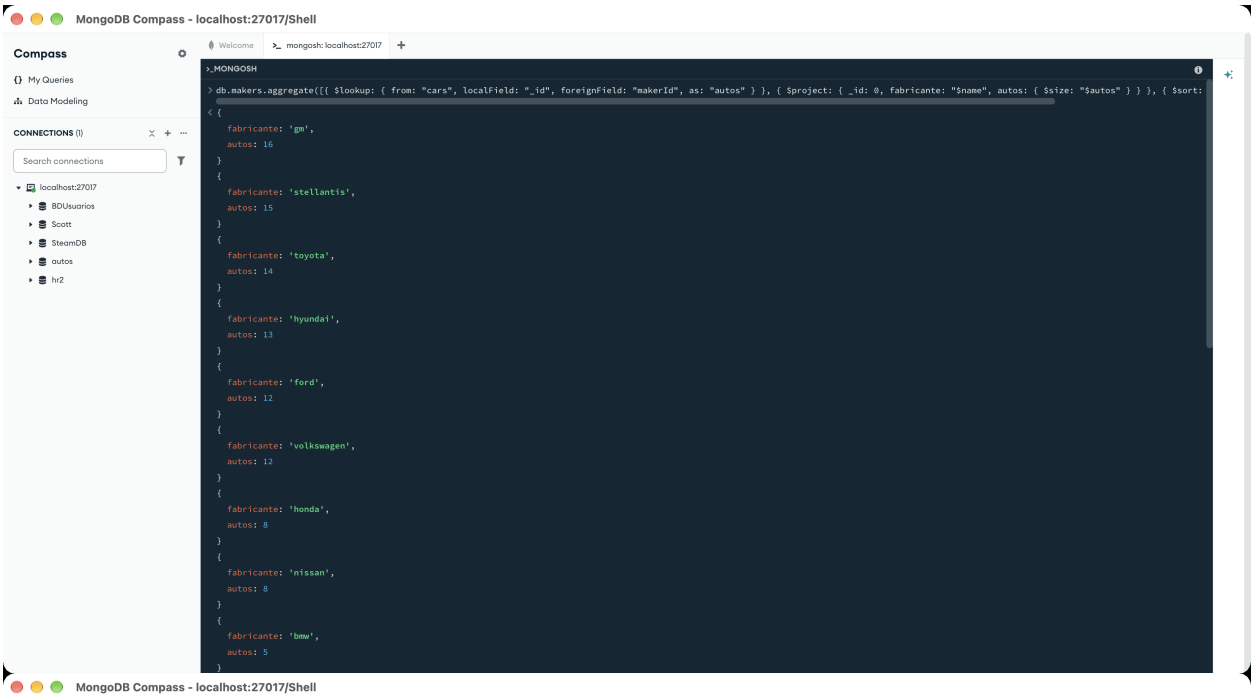


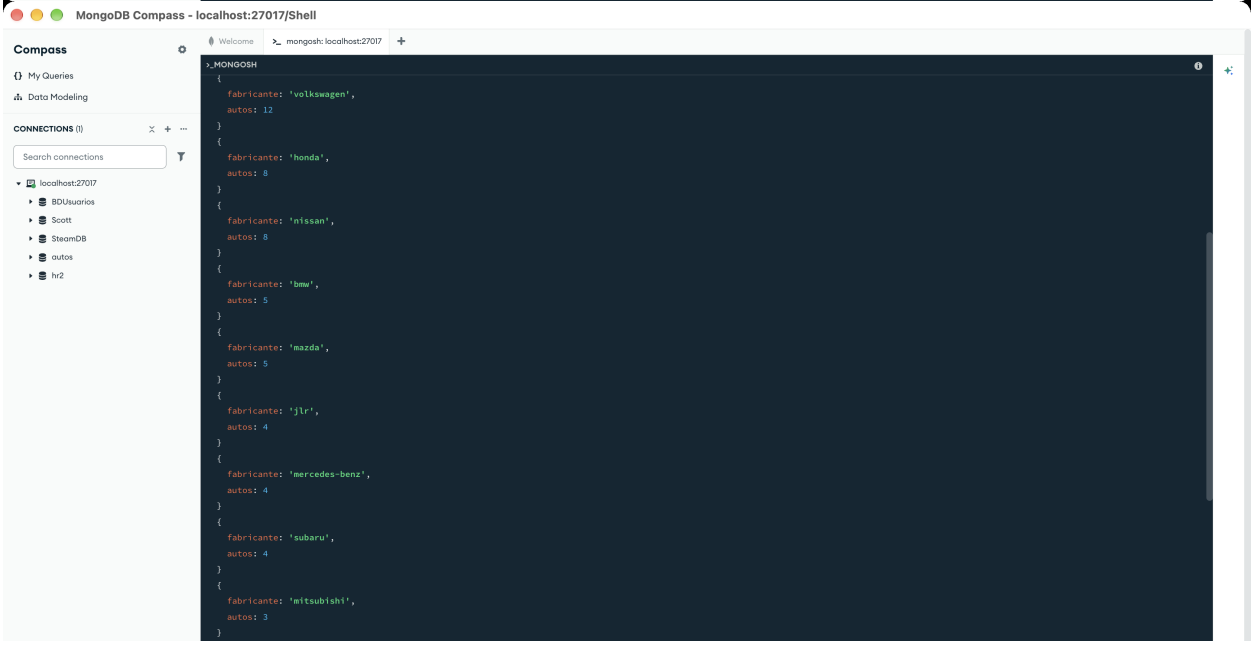


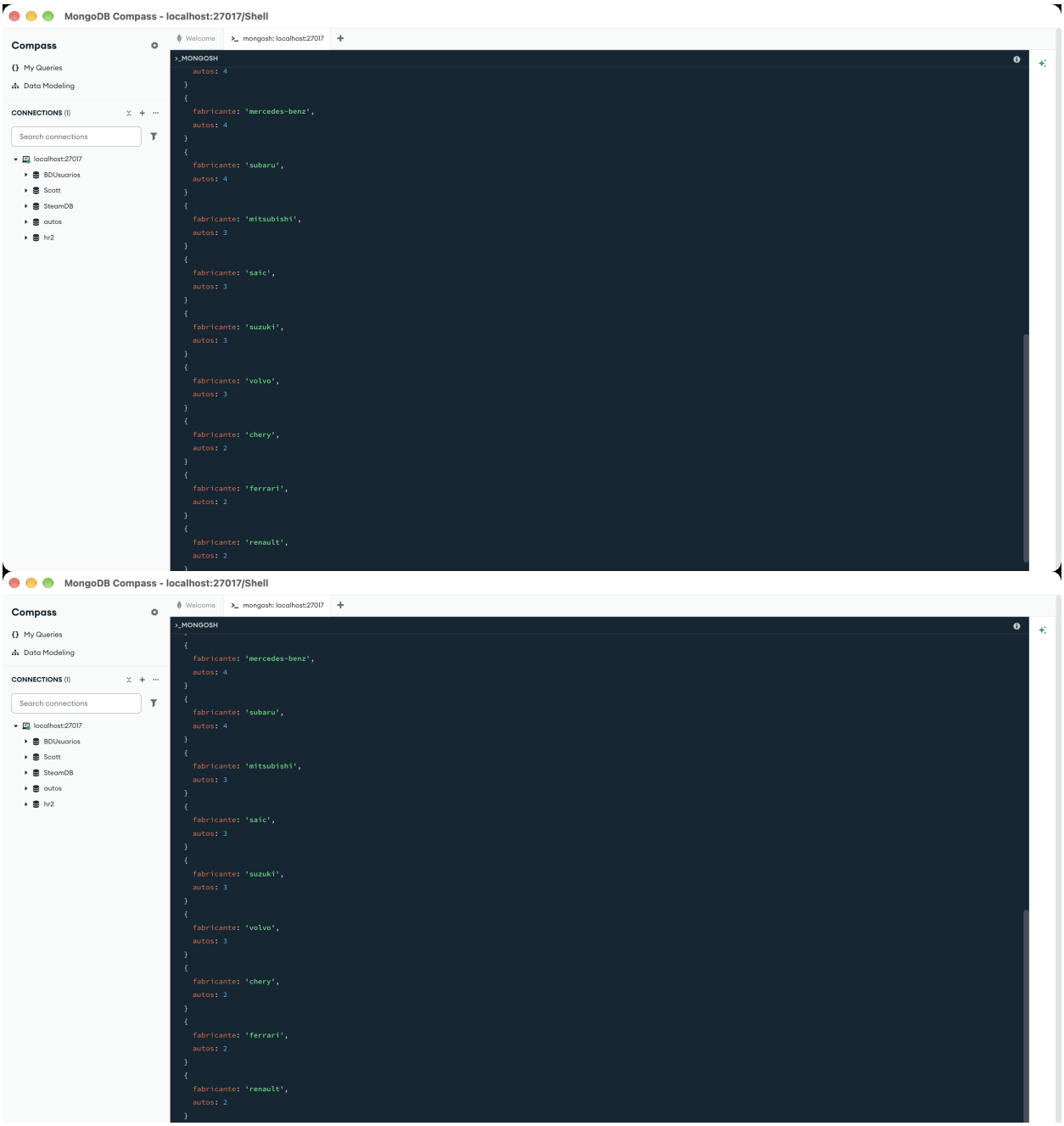
13. Desde la colección makers, obtener cuántos autos tiene registrados cada fabricante (incluidos los que tienen cero).

Método: MakersService.query13(), colección makers

Al partir de makers, un fabricante sin autos queda con un arreglo vacío y cuenta 0.



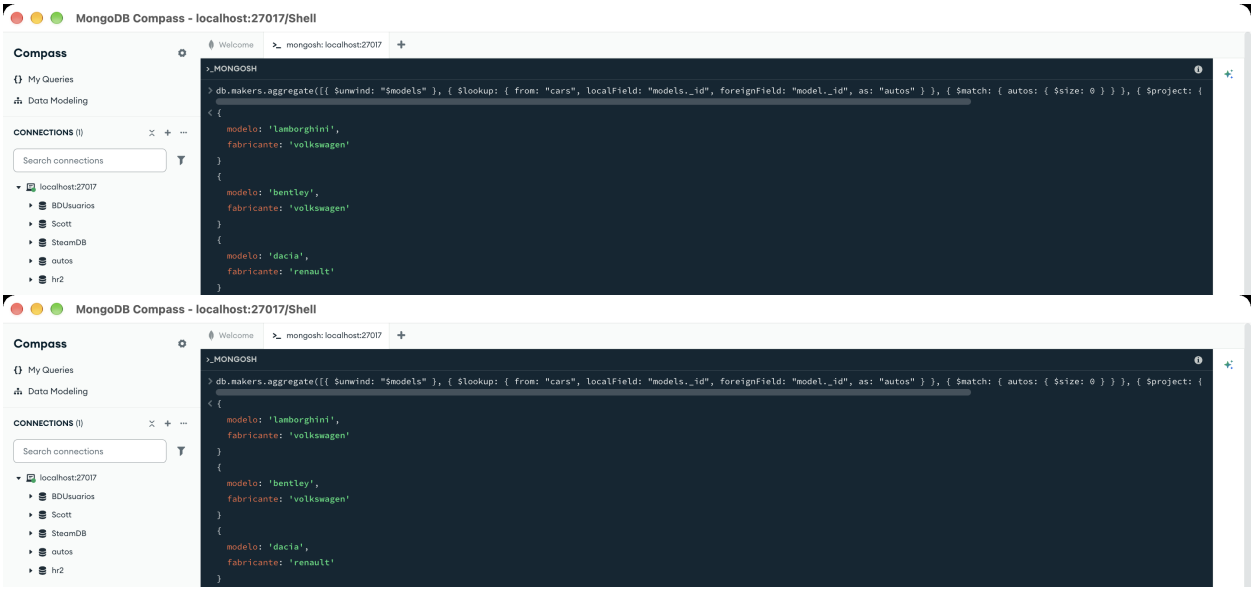




MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
{
fabricante: 'mercedes-benz',
autos: 4
}
{
fabricante: 'subaru',
autos: 4
}
{
fabricante: 'mitsubishi',
autos: 3
}
{
fabricante: 'saic',
autos: 3
}
{
fabricante: 'suzuki',
autos: 3
}
{
fabricante: 'volvo',
autos: 3
}
{
fabricante: 'chery',
autos: 2
}
{
fabricante: 'ferrari',
autos: 2
}
{
fabricante: 'renault',
autos: 2
}

14. Encontrar los modelos del catálogo que no tienen ningún auto registrado.

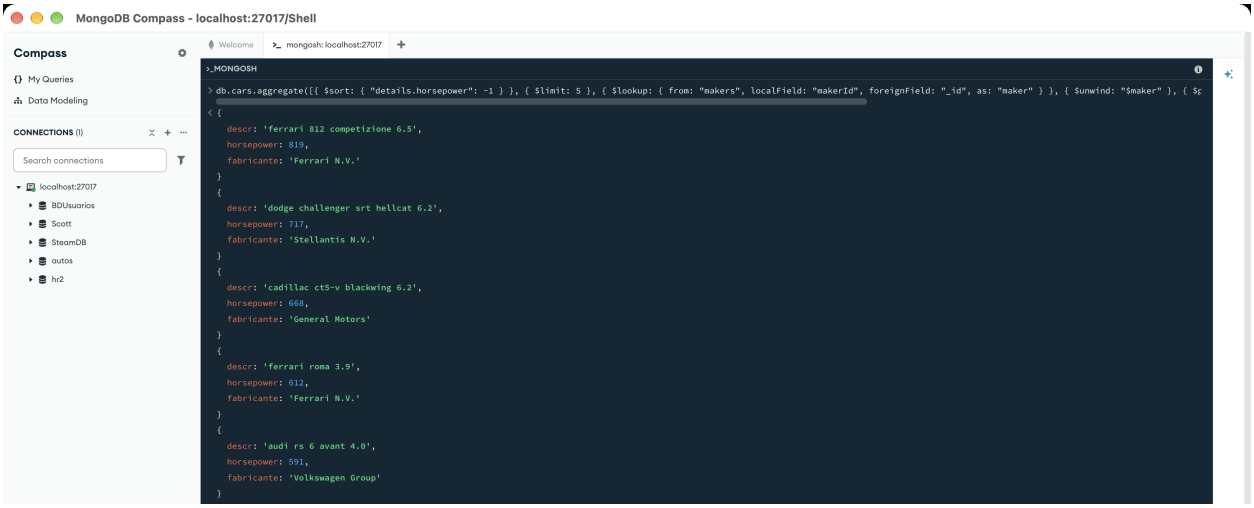
Método: MakersService.query14(), colección makers

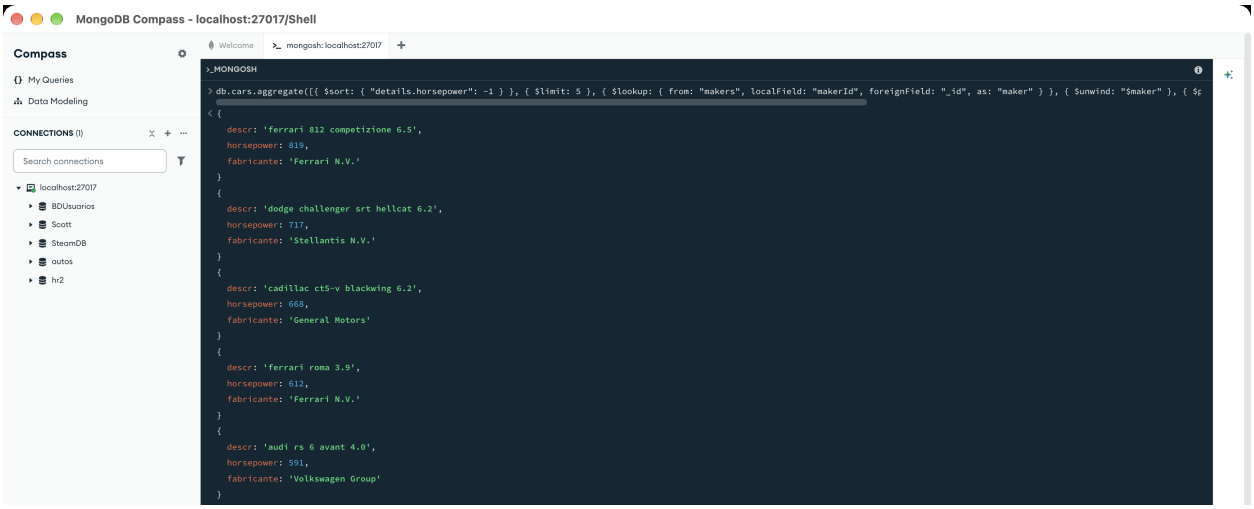


MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
db.makers.aggregate([{ $unwind: "$models" }, { $lookup: { from: "cars", localField: "models._id", foreignField: "model._id", as: "autos" } }, { $match: { autos: { $size: 0 } } }, { $project: {
{
modelo: 'lamborghini',
fabricante: 'volkswagen'
}
{
modelo: 'bentley',
fabricante: 'volkswagen'
}
{
modelo: 'dacia',
fabricante: 'renault'
}

15. Para los 5 autos más potentes, obtener su descripción, potencia y el nombre completo (fullName) de su fabricante.

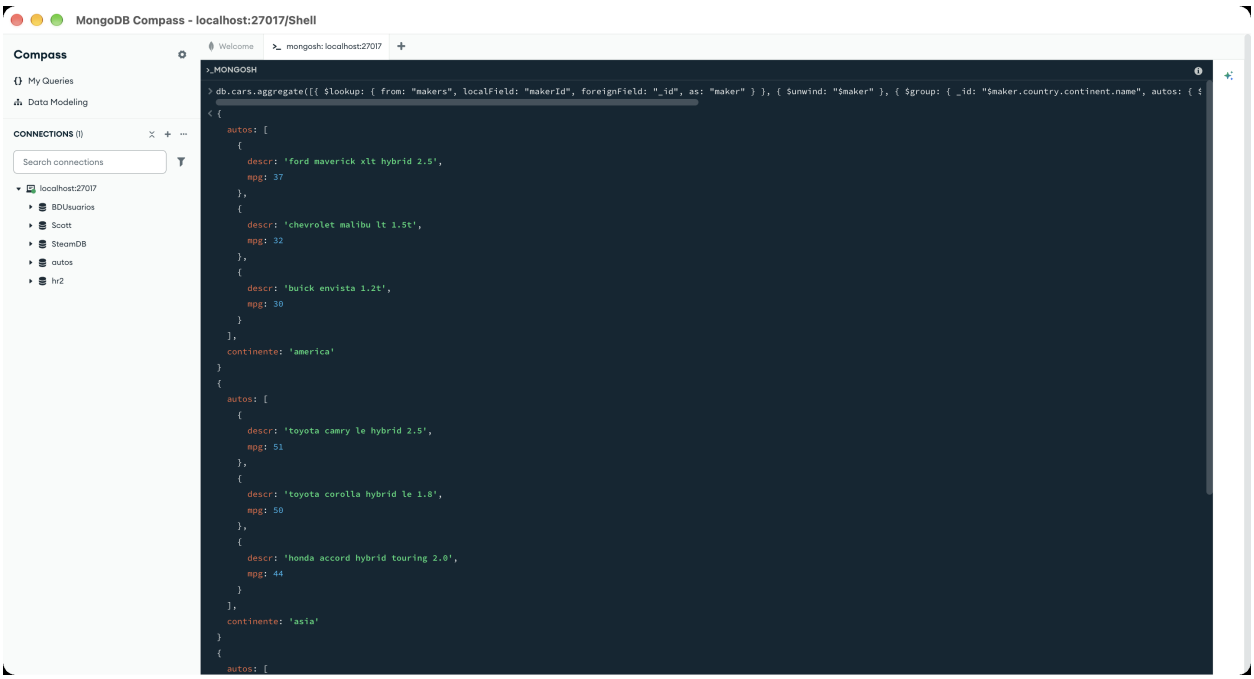
Método: CarsService.query15(), colección cars

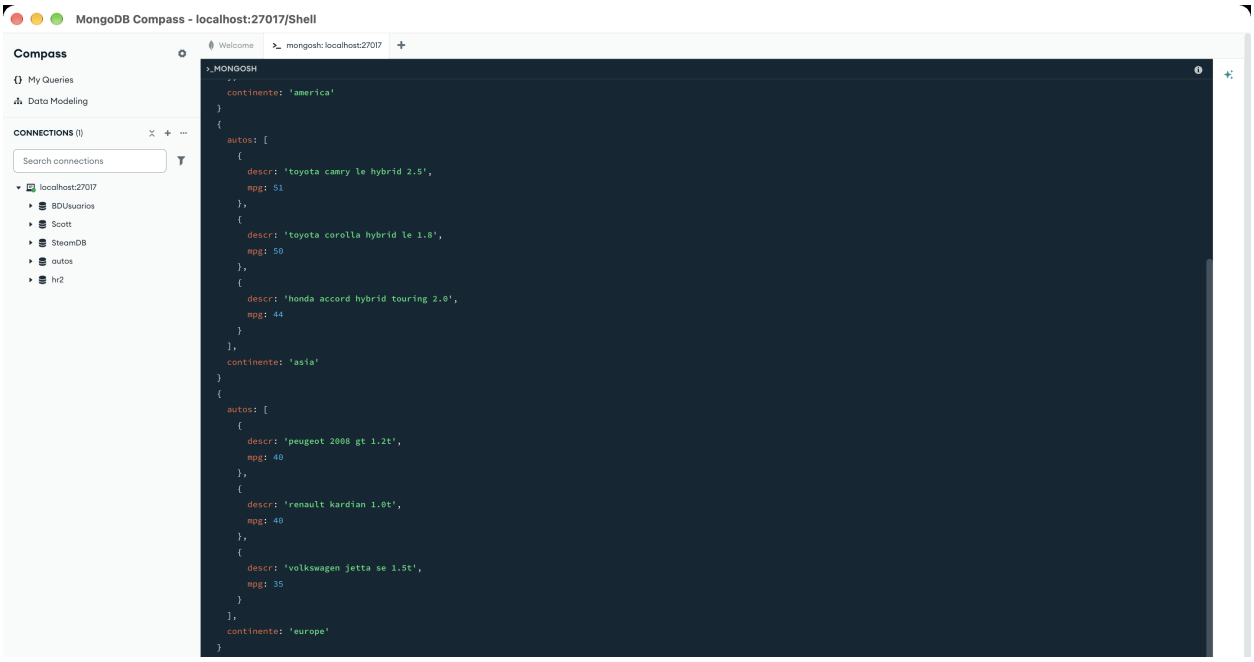




16. Obtener, para cada continente, los 3 autos con mayor rendimiento (descripción y mpg).

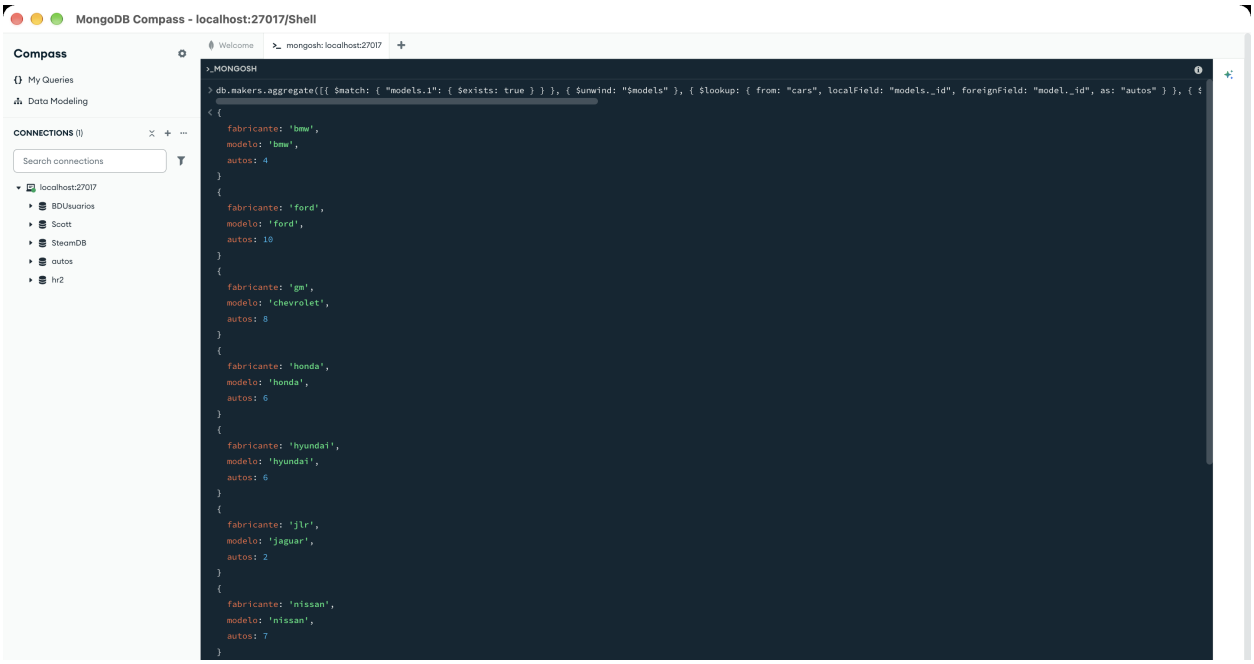
Método: CarsService.query16(), colección cars

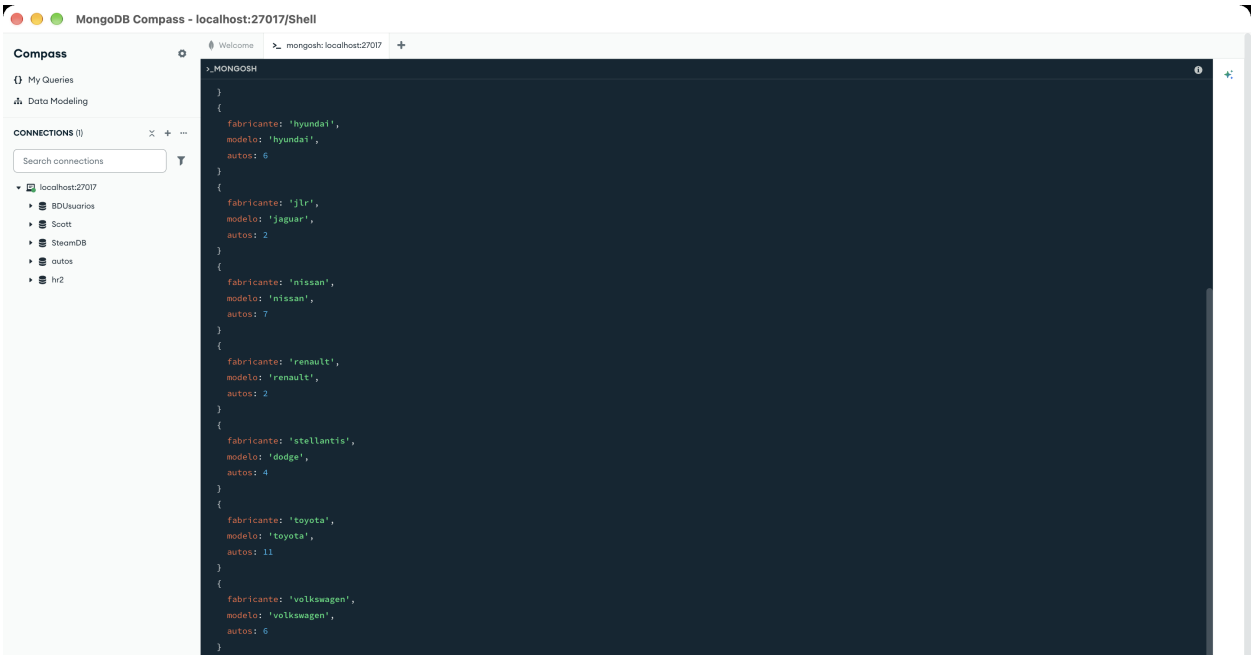




17. Para cada fabricante con más de un modelo, determinar cuál de sus modelos tiene más autos registrados.

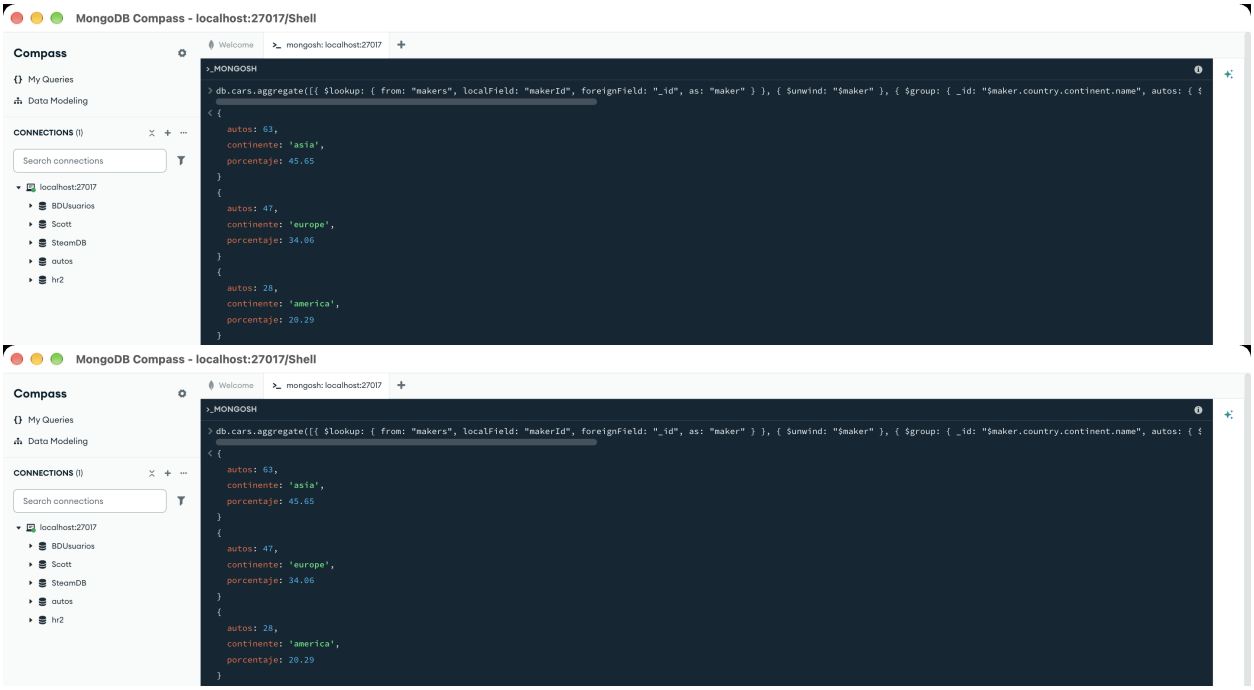
Método: MakersService.query17(), colección makers





18. Calcular el porcentaje que representa cada continente sobre el total de autos, con 2 decimales.

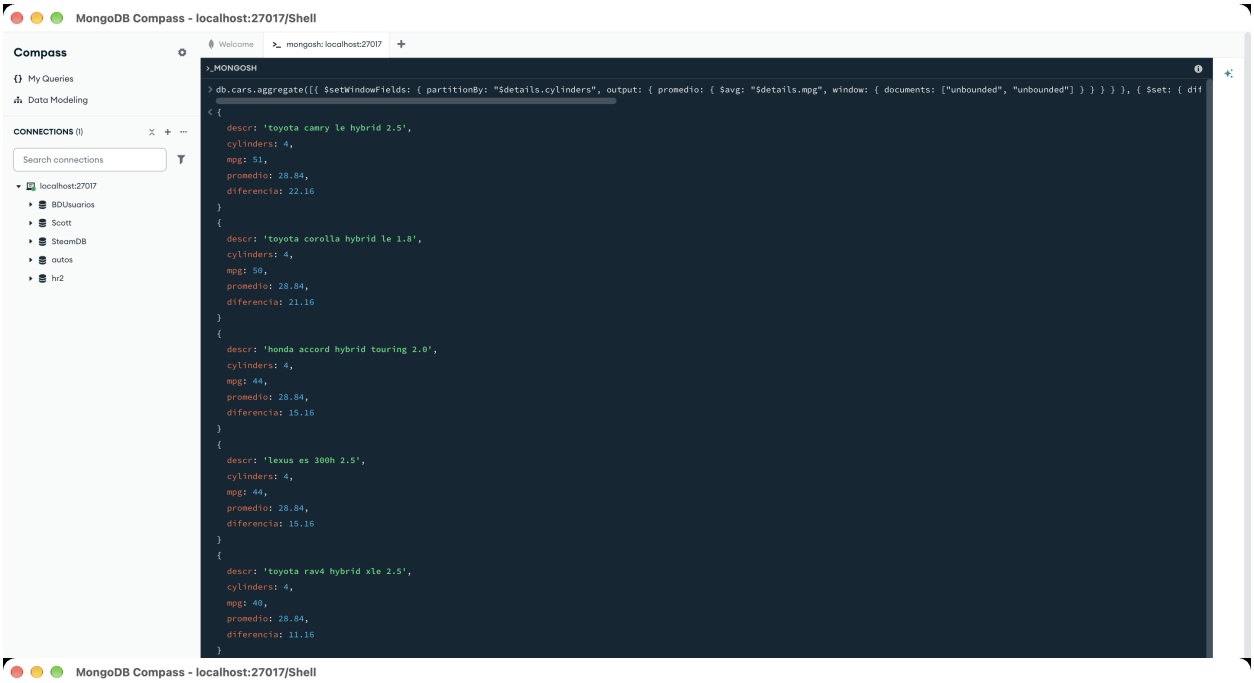
Método: CarsService.query18(), colección cars

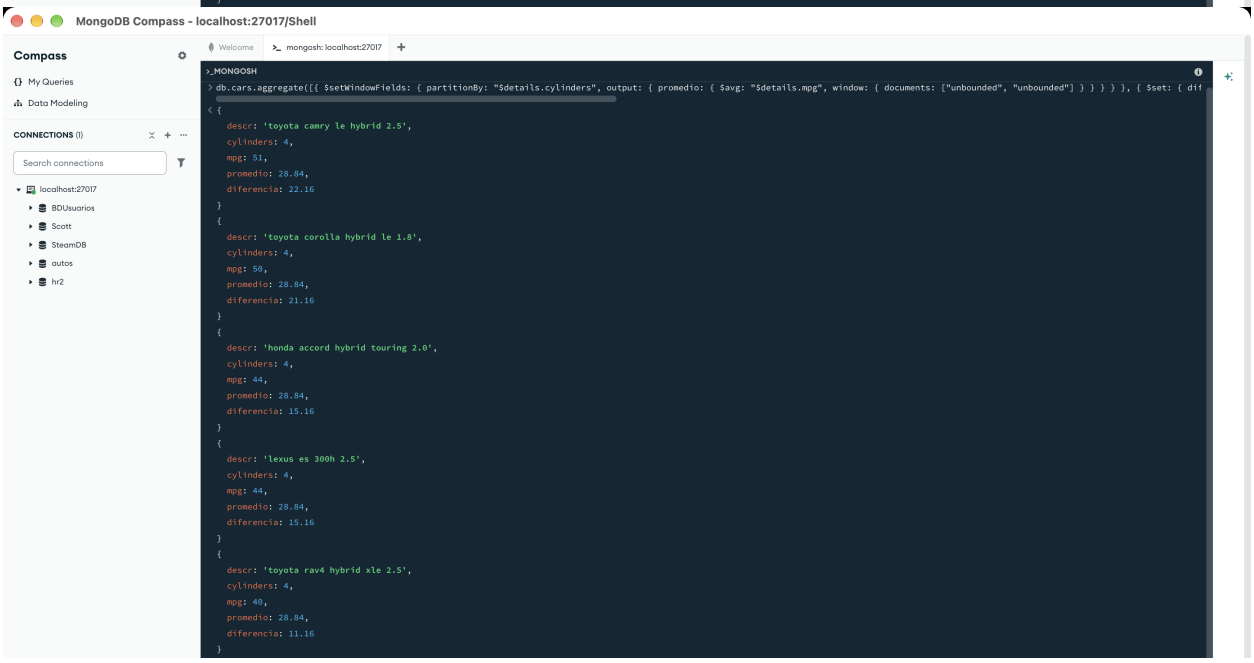


MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
db.cars.aggregate([{ $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as: "maker" } }, { $unwind: "$maker" }, { $group: { _id: "$maker.country.continent.name", autos: { $
{
autos: 63,
continente: 'asia',
porcentaje: 45.65
}
{
autos: 47,
continente: 'europe',
porcentaje: 34.06
}
{
autos: 28,
continente: 'america',
porcentaje: 20.29
}

19. Listar los 5 autos que más superan el rendimiento promedio de los autos con su mismo número de cilindros, mostrando ese promedio y la diferencia.

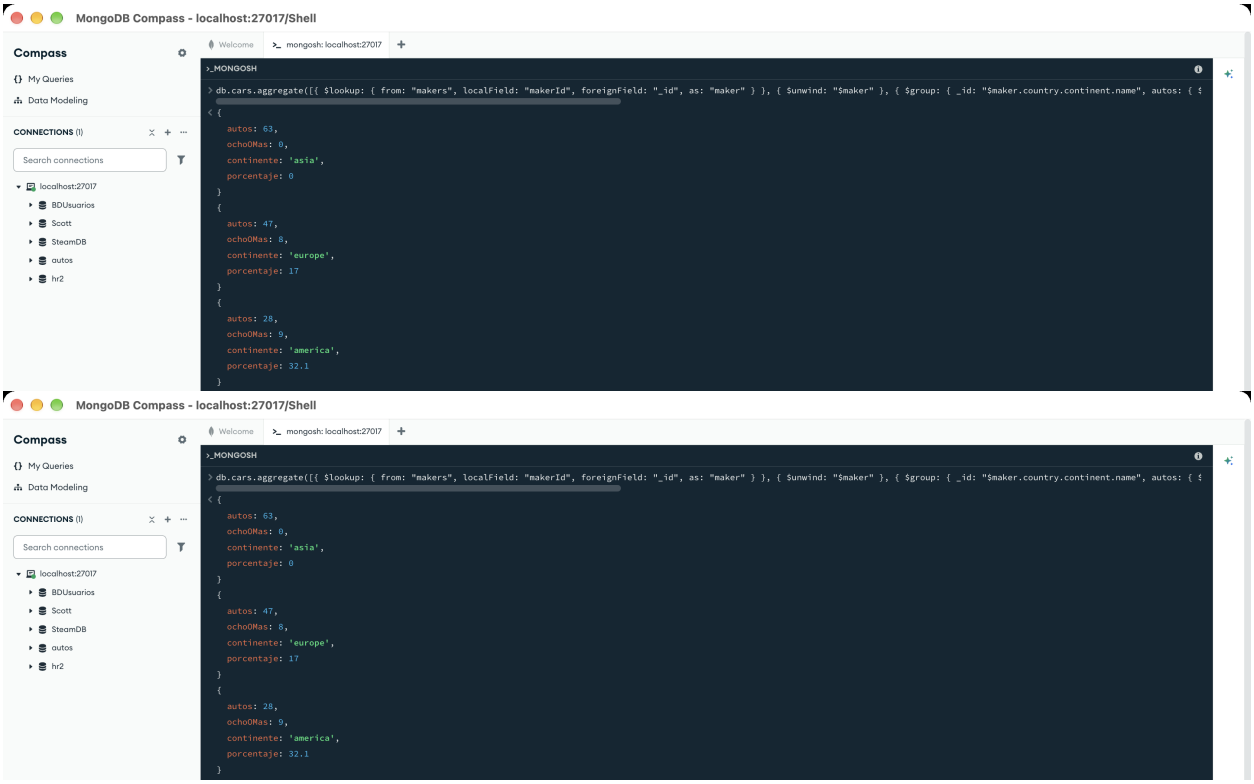
Método: CarsService.query19(), colección cars





20. Calcular, por continente, el número de autos, cuántos tienen 8 o más cilindros y el porcentaje que representan (1 decimal).

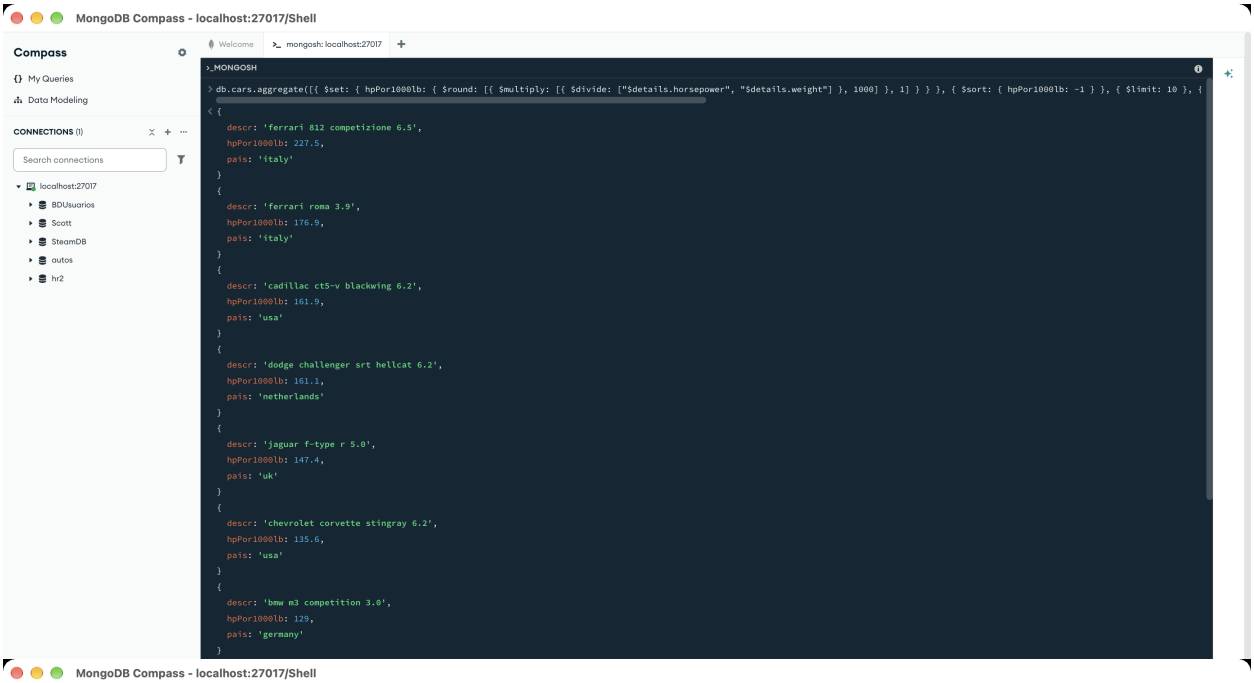
Método: CarsService.query20(), colección cars

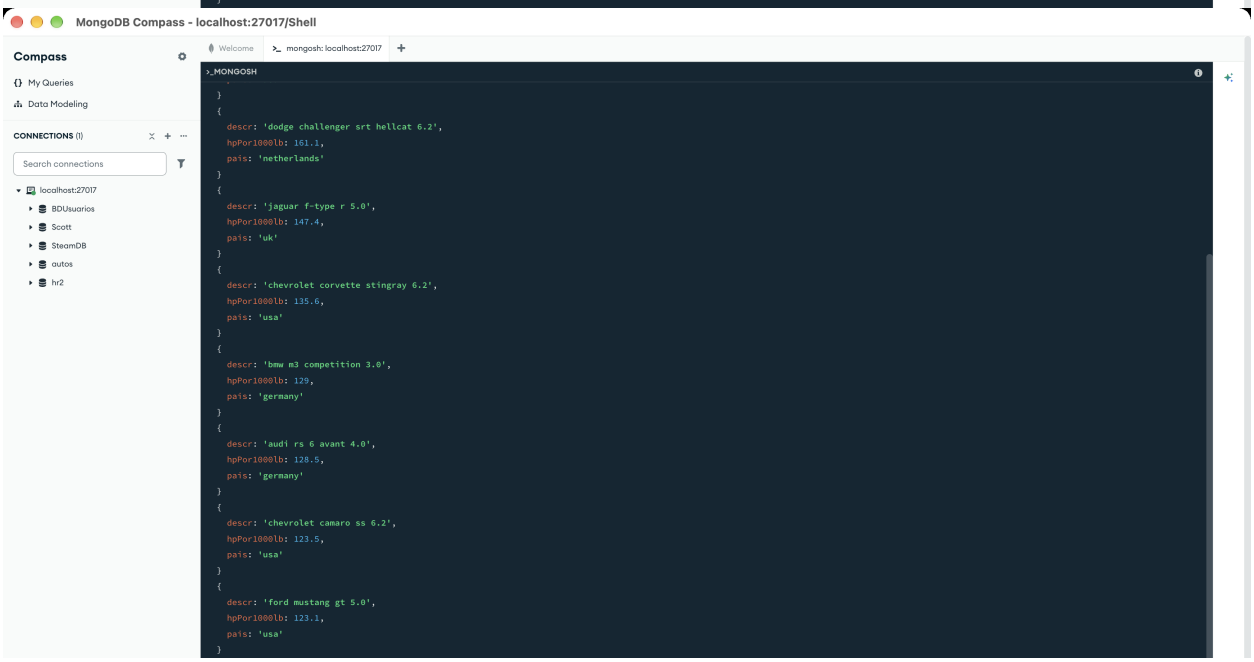


MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
db.cars.aggregate([{ $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as: "maker" } }, { $unwind: "$maker" }, { $group: { _id: "$maker.country.continent.name", autos: { $
{
autos: 63,
ochoOMas: 0,
continente: 'asia',
porcentaje: 0
}
{
autos: 47,
ochoOMas: 8,
continente: 'europe',
porcentaje: 17
}
{
autos: 28,
ochoOMas: 9,
continente: 'america',
porcentaje: 32.1
}

21. Calcular la potencia por cada 1000 lb de peso y mostrar los 10 autos con mayor valor, junto con su país.

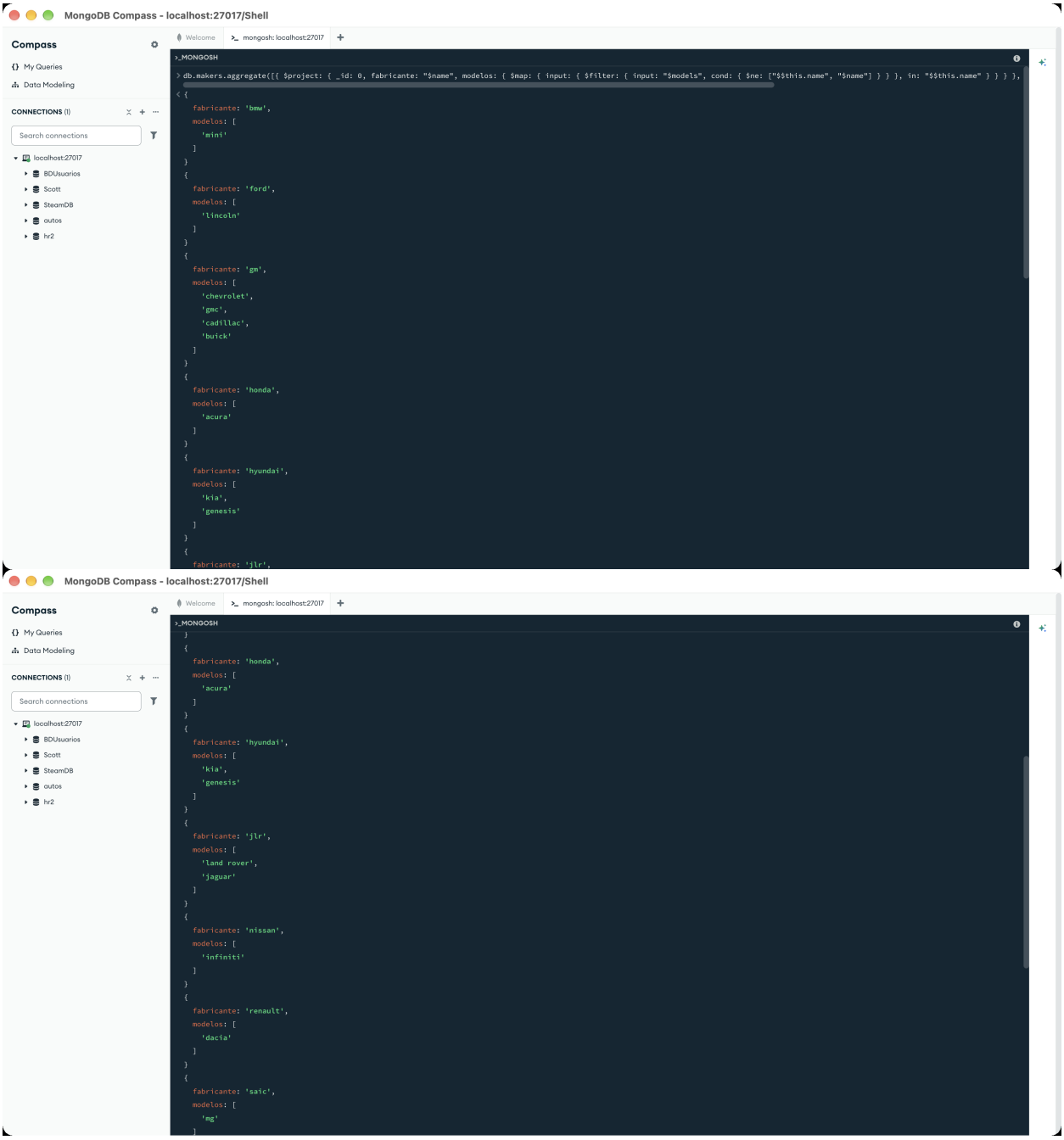
Método: CarsService.query21(), colección cars

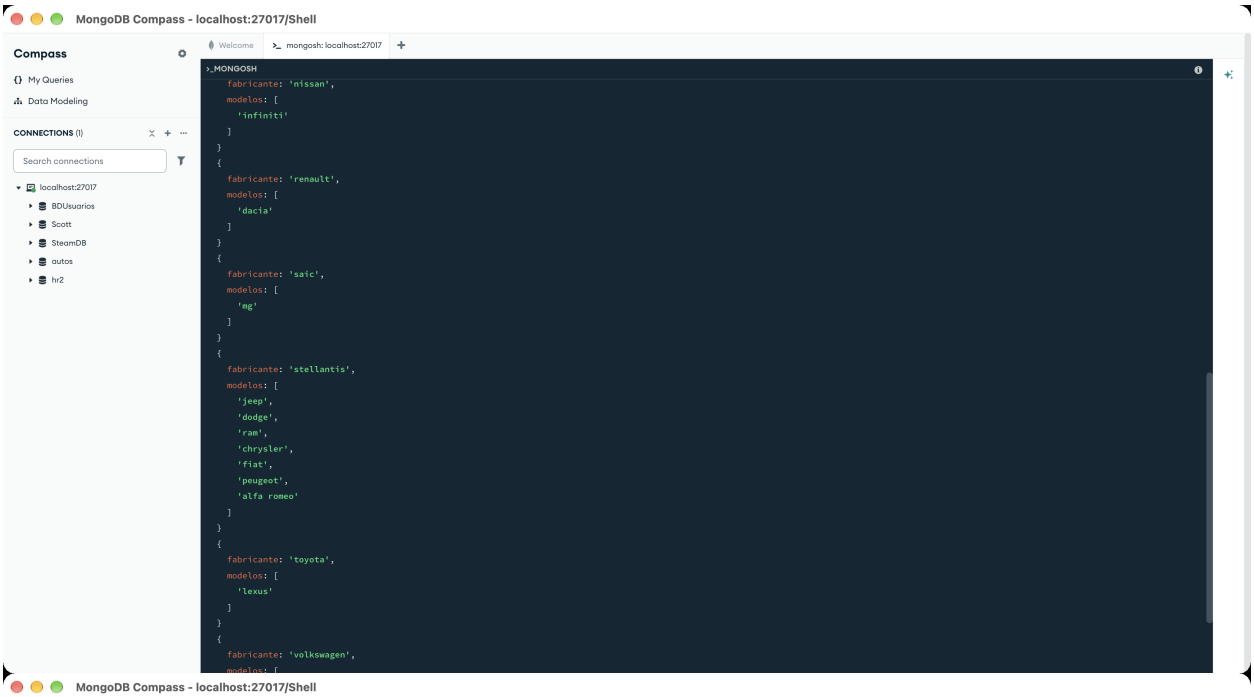


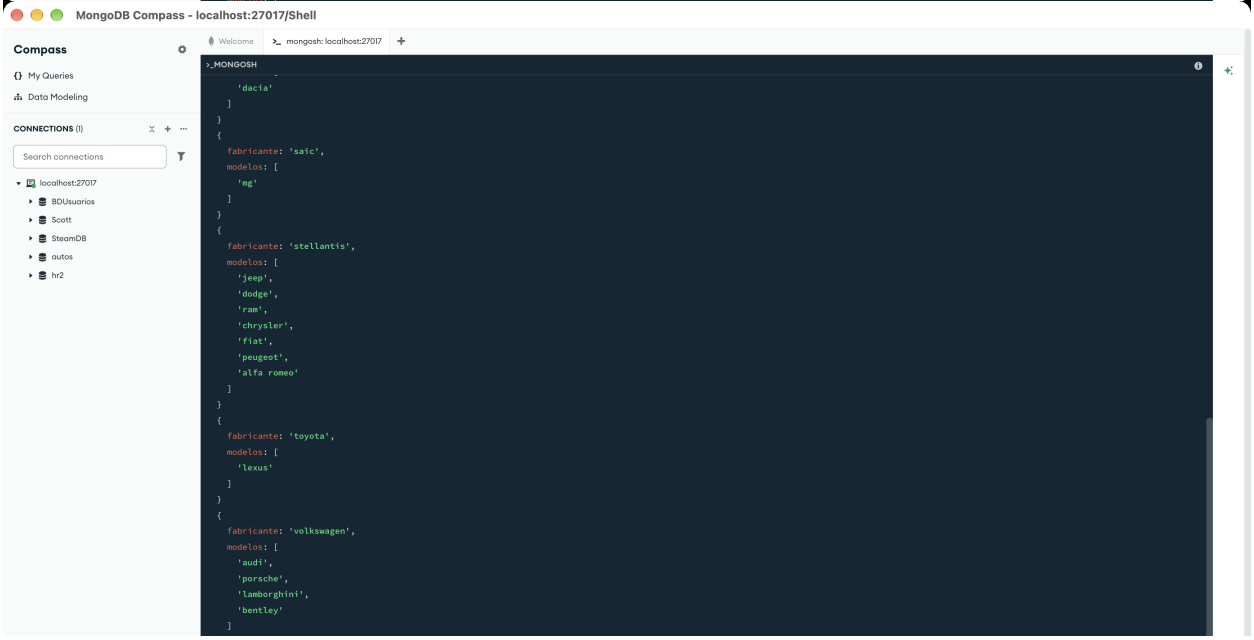


22. Para cada fabricante, mostrar solo los modelos cuyo nombre es distinto al nombre del fabricante, descartando a los fabricantes que no tengan ninguno.

Método: MakersService.query22(), colección makers

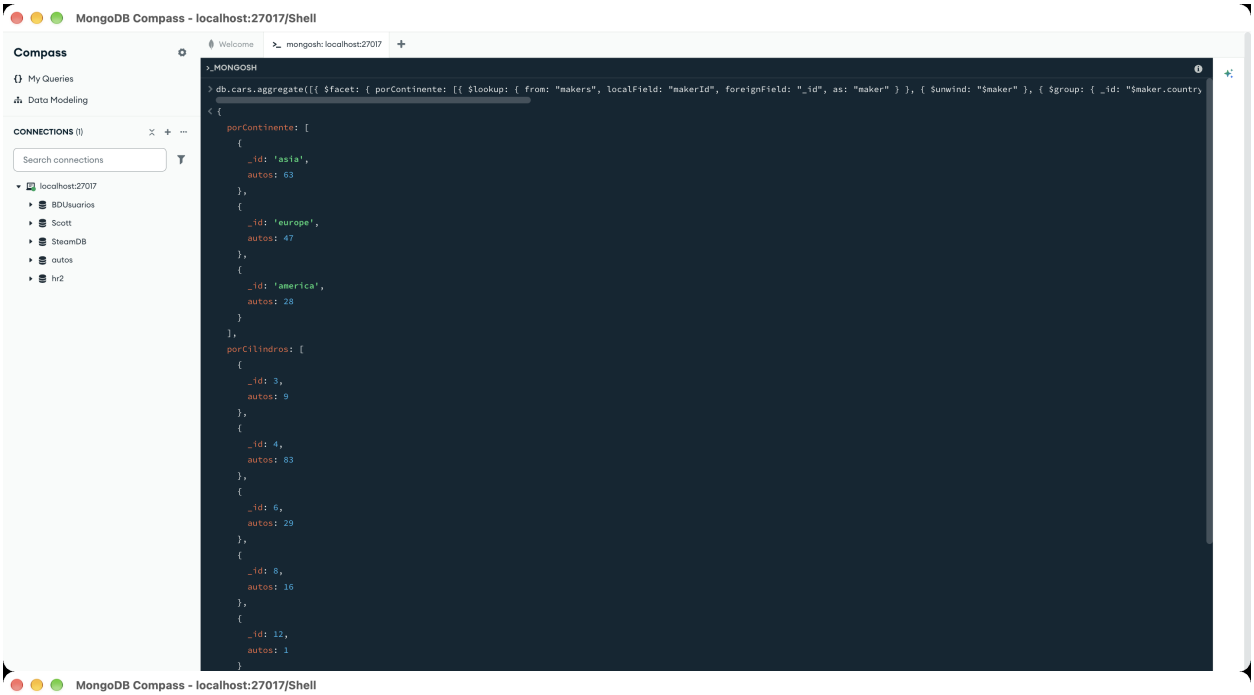


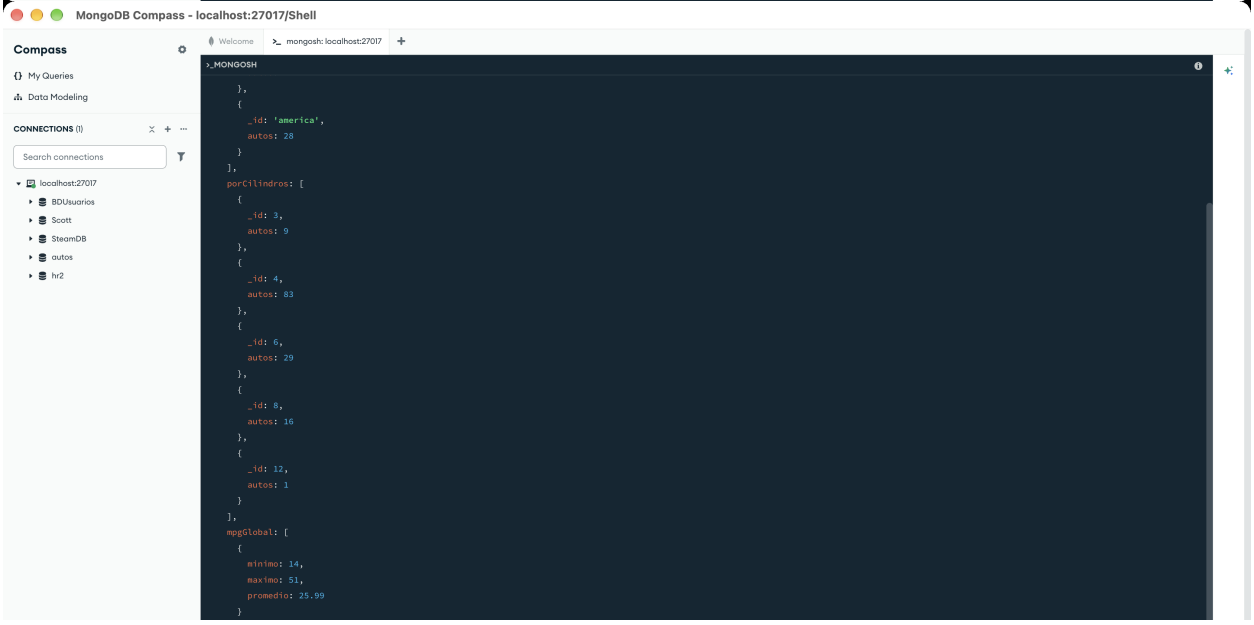

MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
fabricante: 'nissan',
modelos: [
'infiniti'
]
}
{
fabricante: 'renault',
modelos: [
'dacia'
]
}
{
fabricante: 'saic',
modelos: [
'mg'
]
}
{
fabricante: 'stellantis',
modelos: [
'jeep',
'dodge',
'ram',
'chrysler',
'fiat',
'peugeot',
'alfa romeo'
]
}
{
fabricante: 'toyota',
modelos: [
'lexus'
]
}
{
fabricante: 'volkswagen',


MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
'dacia'
]
}
{
fabricante: 'saic',
modelos: [
'mg'
]
}
{
fabricante: 'stellantis',
modelos: [
'jeep',
'dodge',
'ram',
'chrysler',
'fiat',
'peugeot',
'alfa romeo'
]
}
{
fabricante: 'toyota',
modelos: [
'lexus'
]
}
{
fabricante: 'volkswagen',
modelos: [
'audi',
'porsche',
'lamborghini',
'bentley'
]

23. En una sola consulta, obtener: (a) el número de autos por continente, (b) el número de autos por cilindros y (c) las estadísticas globales de mpg (mínimo, máximo y promedio).

Método: CarsService.query23(), colección cars

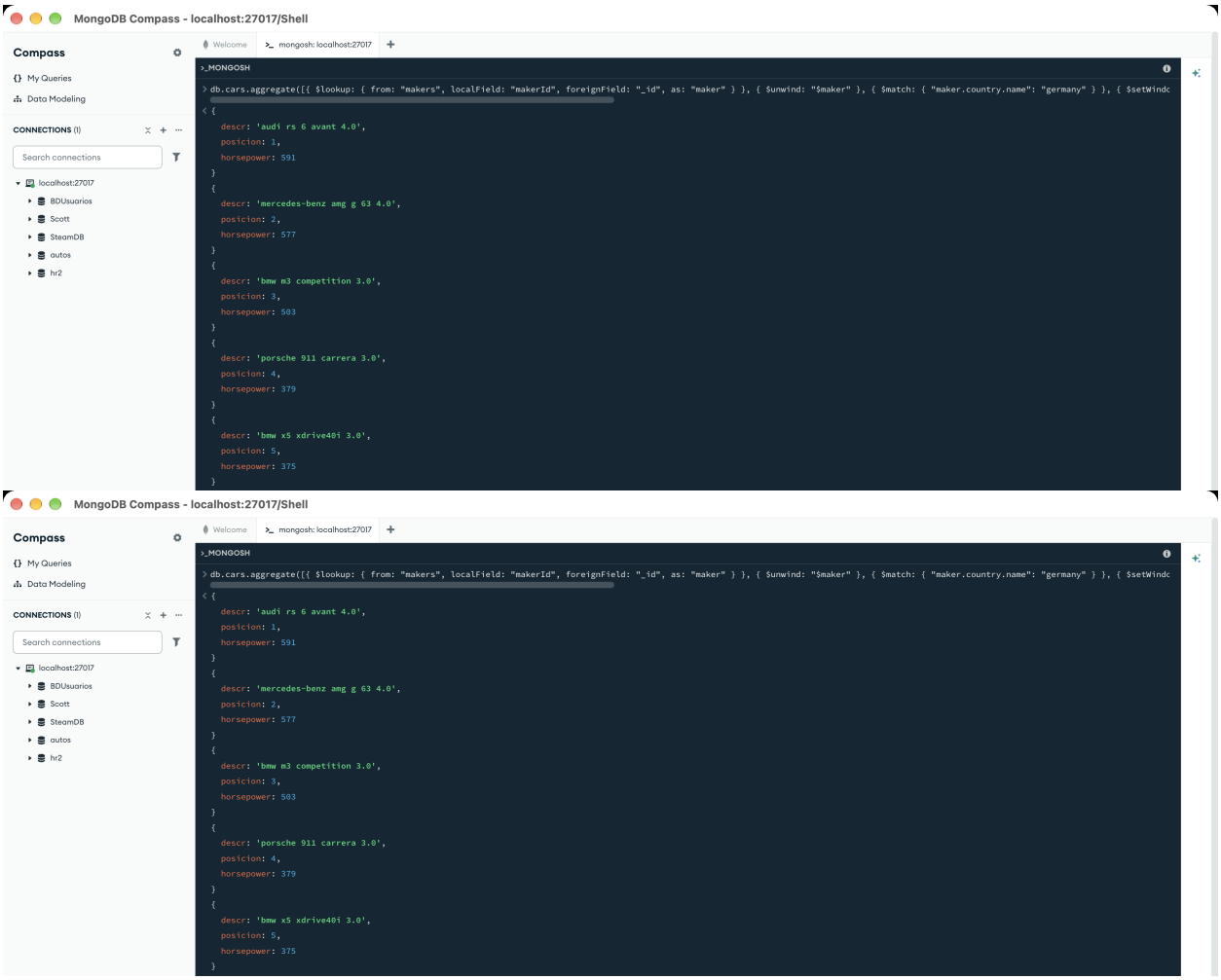




24. Asignar a cada auto alemán su posición por potencia dentro de su país y mostrar los 5 primeros.

Método: CarsService.query24(), colección cars

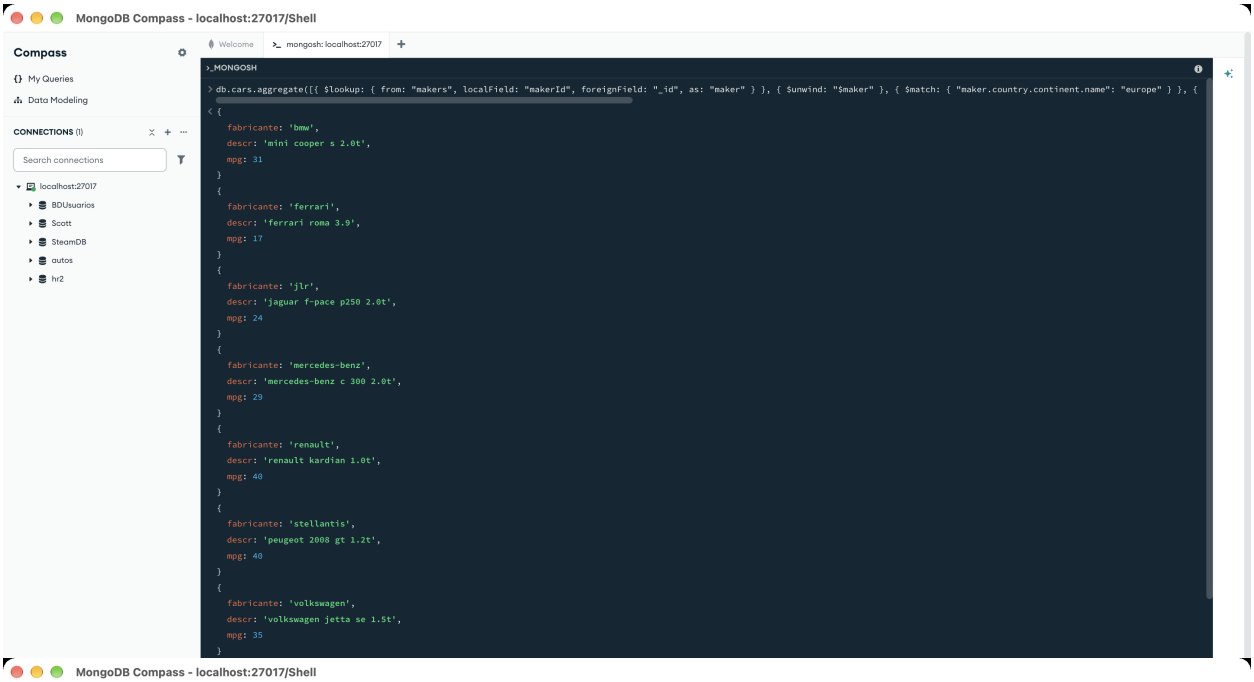
$rank da la misma posición a los empates y salta la siguiente.

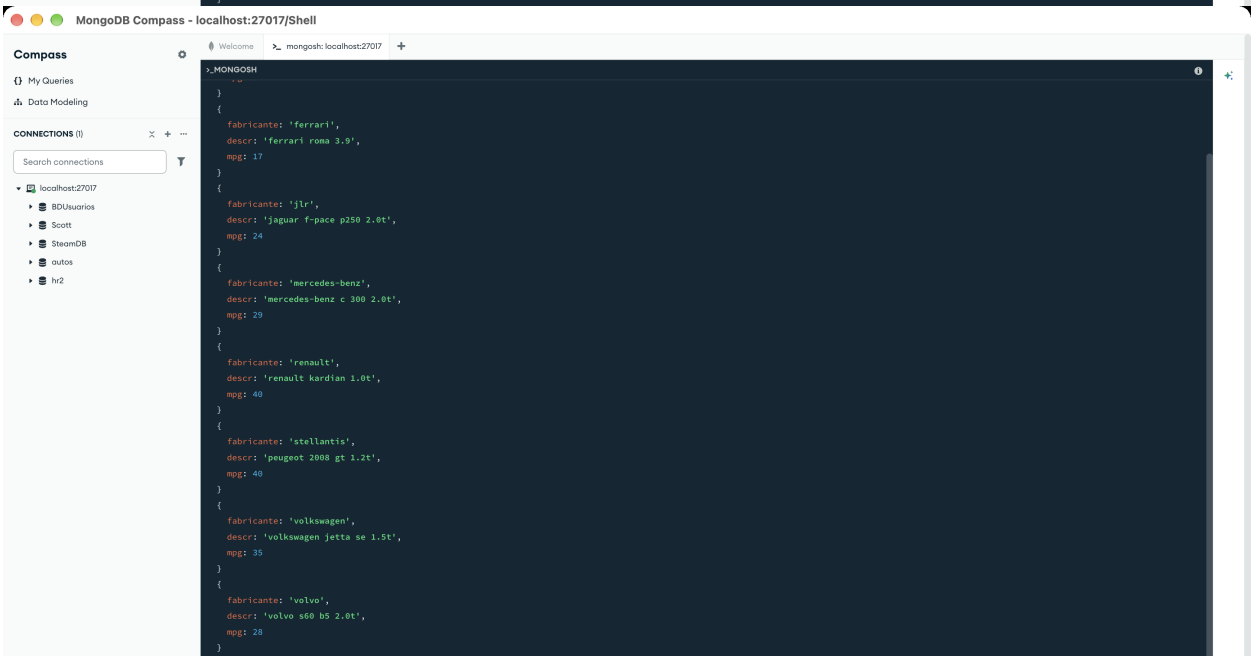


MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
BDUsuarios
Scott
SteamDB
autos
hr2
Welcome
mongosh: localhost:27017
>_MONGOSH
db.cars.aggregate([{ $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as: "maker" } }, { $unwind: "$maker" }, { $match: { "maker.country.name": "germany" } }, { $setWindc
{
descr: 'audi rs 6 avant 4.0',
posicion: 1,
horsepower: 591
}
{
descr: 'mercedes-benz amg g 63 4.0',
posicion: 2,
horsepower: 577
}
{
descr: 'bmw m3 competition 3.0',
posicion: 3,
horsepower: 503
}
{
descr: 'porsche 911 carrera 3.0',
posicion: 4,
horsepower: 379
}
{
descr: 'bmw x5 xdrive40i 3.0',
posicion: 5,
horsepower: 375
}

25. Obtener para cada fabricante europeo su auto más eficiente.

Método: CarsService.query25(), colección cars





IV. Aplicación en Python

La aplicación está hecha con Flask y PyMongo y está organizada como un proyecto de NestJS: cada colección es un módulo con su controller, su service y su schema.

```
main.py                     arranca la app
app/__init__.py             crea la app y registra los controllers
app/database.py             conexión a MongoDB
app/modules/
  cars/                     cars_controller.py, cars_service.py (19 consultas),
                            schemas/car_schema.py
  makers/                   makers_controller.py, makers_service.py (5 consultas),
                            schemas/maker_schema.py
  continents/               continents_controller.py, continents_service.py (consulta 9),
                            schemas/continent_schema.py
  dashboard/                resumen de las colecciones
app/templates/              query.html (código y resultado), menú de las 25
scripts/                    migrate.py y export_queries.py
database/                   autos_mongodb.js y queries.js
tests/                      test_app.py
```

El service tiene un método por consulta. El método arma el pipeline, lo ejecuta en su colección y regresa el pipeline junto con el resultado:

```
class MakersService:
    def __init__(self):
        self.makers = get_db()["makers"]

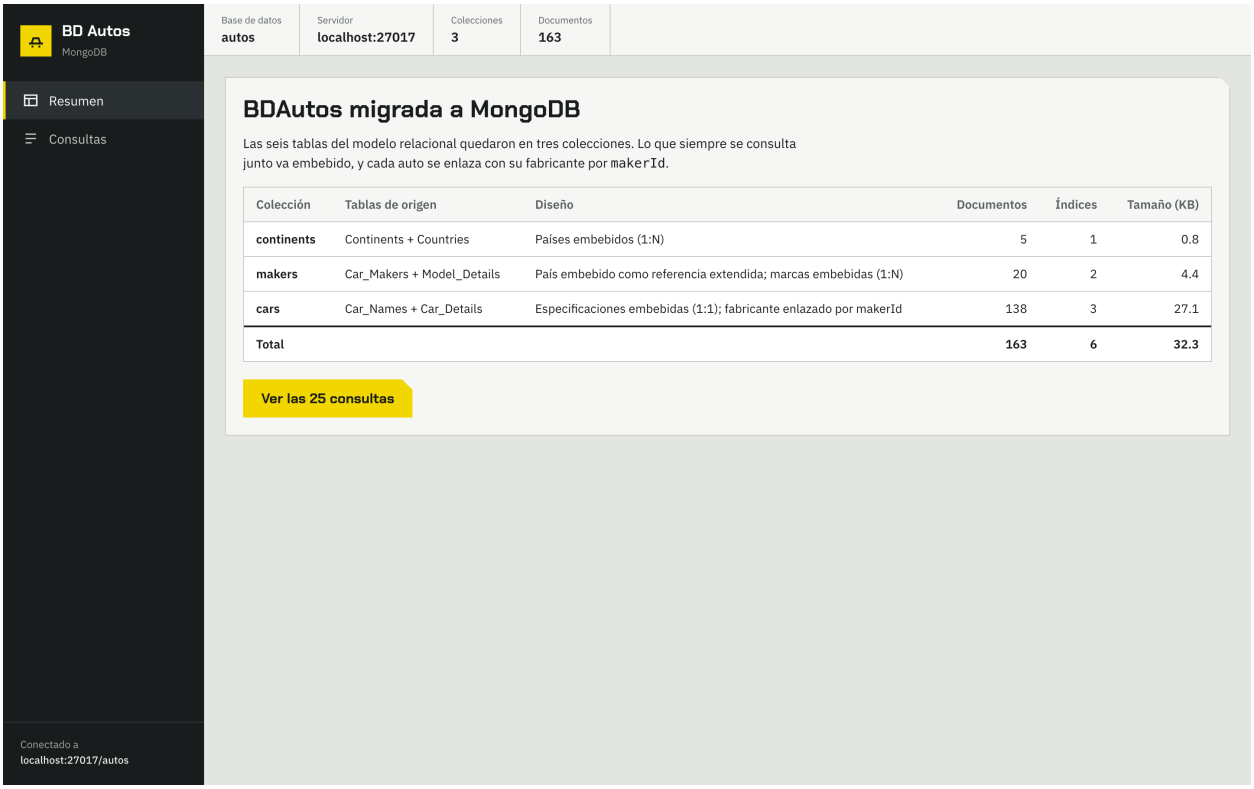
    def query5(self):
        """5. Obtener los fabricantes que comercializan más de una marca o línea de modelo,
        mostrando nombre, país y modelos."""
        pipeline = [
            MORE_THAN_ONE_MODEL,
            {"$project": {"_id": 0, "name": 1, "country": "$country.name",
                          "models": "$models.name"}},
            {"$sort": {"name": 1}},
        ]
        return pipeline, list(self.makers.aggregate(pipeline))
```

El controller tiene una ruta por consulta, llama al service y manda el código y el resultado a la página:

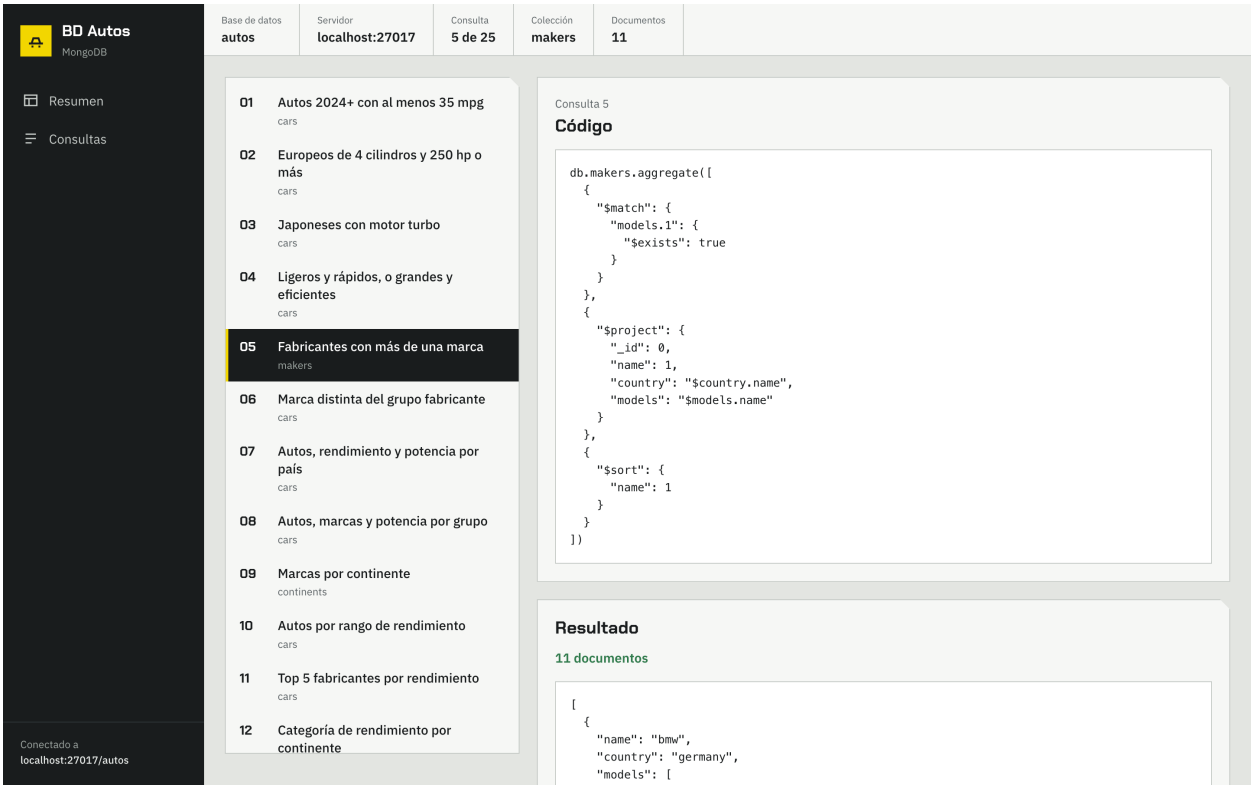
```
@makers.get("/queries/5")
def query5():
    pipeline, result = service.query5()
    return render_template("query.html", number=5, collection="makers",
                           pipeline=pipeline, result=result)
```

Para correrla: .venv/bin/python -m scripts.migrate carga la base y .venv/bin/python main.py abre la app en http://127.0.0.1:5000. En el menú de la izquierda están las 25 consultas; cada una muestra su código db.<colección>.aggregate(...) y el resultado en JSON.

Resumen de la base de datos



Consulta 5: código y resultado



Consulta 23: código con $facet

